

MATH 8090: ARMA Models: Prediction and Forecasting, Modeling Case Study

Whitney Huang, Clemson University

9/28-10/2/2026

Contents

NOAA wind data example	2
Load and plot the data	2
“Estimate” ϕ using sample ACF and center the data	3
One-step-ahead forecast	5
Fill in missing value example	7
Simulate an AR(-0.9)	7
Let’s remove some data to illustrate how to fill in missing values using forecasting algorithm	9
Fill in “missing” values	10
Prediction Errors from Best Linear Predictor	11
Ireland wind data case study	12
Load and plot the data	12
Deseasonalization: Harmonic Regression	13
ACF Plots: Original and Deseasonalized Series	15
Apply transformation to make wind speed more Gaussian like	16
Now take square roots of the original data and deseasonalizeagain!	17
Checking Normality ACF/PACF	18
Model identification, fitting, and selection	19
Let’s first fit an AR(1)	19
Fit an AR(2) model	22
Fit an ARMA(1,1) model	25
Use AIC to conduct model selection	31
Forecasting	32
Visualizing the Forecasts	33

NOAA wind data example

This example is taken from Don Percival's time series course (UW Stat 519).

The one-step-ahead forecast of an AR(1) process is:

$$P_n X_{n+1} = \hat{\mu} + \hat{\phi}(X_n - \hat{\mu}),$$

where $\hat{\phi}$ is our estimate of ϕ , and $\hat{\mu}$ is an estimate of μ .

Load and plot the data

```
# Read the wind speed time series from the online data file
ws <- scan("http://faculty.washington.edu/dbp/s519/Data/wind-speed.txt")

# Determine the number of observations
n <- length(ws)

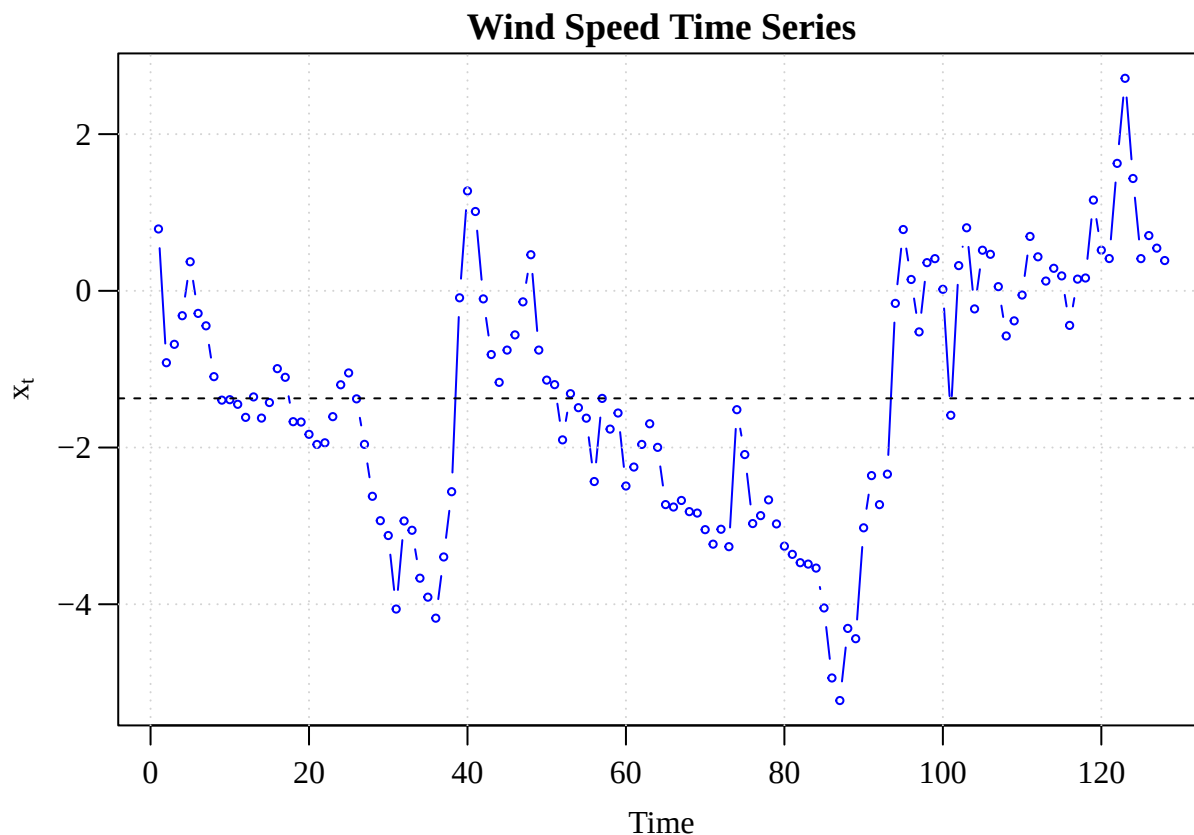
# Set graphical parameters for the time series plot
par(las = 1, mgp = c(2, 0.7, 0), mar = c(3.6, 3.6, 1.4, 0.6),
    family = "serif")

# Plot the wind speed observations over time
plot(ws, col = "blue", xlab = "Time", type = "b",
     ylab = expression(x[t]), main = "Wind Speed Time Series", cex = 0.5)

# Add grid lines to the plot
grid()

# Calculate the sample mean of the wind speed series
xbar_ws <- mean(ws)

# Add the sample mean as a horizontal dashed line
abline(h = xbar_ws, lty = 2)
```



“Estimate” ϕ using sample ACF and center the data

```
# --- Estimate the AR(1) coefficient using the lag-1 sample ACF -----
# For an AR(1) process, rho(1) = phi, so the lag-1 sample ACF provides
# a simple moment-based estimate of the AR coefficient
acf.ws <- acf(ws, lag.max = 40, plot = FALSE)$acf # includes lag 0
phi.ws <- acf.ws[2]                               # lag-1 sample ACF

# --- Bartlett variance factor for the sample ACF under an AR(1) model ----
# Compute w_hh(phi), where the approximate sampling variance is
# Var(rho_hat(h)) w_hh(phi) / n
gen.whh.ar <- function(h, phi){
  p2 <- phi^2
  p2h <- p2^h
  -2 * h * p2h + (1 - p2h) * (1 + p2) / (1 - p2)
}

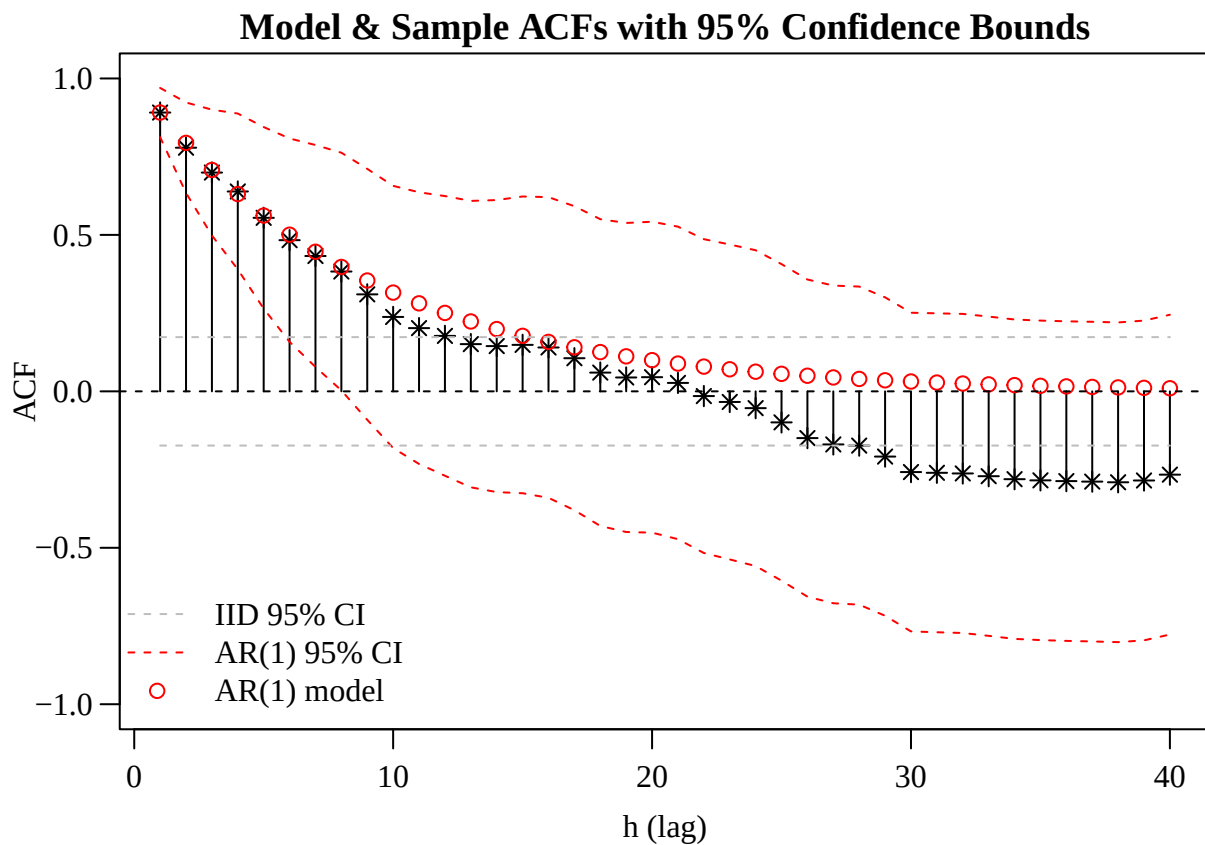
# --- Compare the sample ACF with the fitted AR(1) ACF -----
# Plot the sample ACF together with:
# 1. the theoretical ACF implied by the fitted AR(1) model,
# 2. Bartlett-based 95% confidence bounds under the AR(1) model, and
# 3. the usual IID 95% confidence bounds for comparison
plot.ACFbartlettAR <- function(ts, n.lags = 40){
  stopifnot(is.numeric(ts), length(ts) > 5)
  n.ts <- length(ts)
```

```

lags <- 1:n.lags
# Compute the sample ACF for lags 1,...,n.lags (excluding lag 0)
acf.full <- acf(ts, lag.max = n.lags, plot = FALSE)$acf
acf.est <- acf.full[-1]
# Estimate phi by the lag-1 sample ACF and obtain the corresponding
# theoretical AR(1) ACF: rho(h) = phi^h
phi.hat <- acf.est[1]
acf.model <- phi.hat^lags
# Plot the sample ACF using vertical stems and star-shaped points
plot(lags, acf.est, type = "h",
     xlab = "h (lag)", ylab = "ACF",
     ylim = c(-1, 1),
     main = "Model & Sample ACFs with 95% Confidence Bounds",
     las = 1)
points(lags, acf.est, pch = 8)
# Add the theoretical ACF implied by the fitted AR(1) model
points(lags, acf.model, col = "red")
# Add Bartlett-based 95% confidence bounds under the AR(1) model
CI.AR <- 1.96 * sqrt(sapply(lags, function(h)
  gen.whh.ar(h, phi.hat))) / sqrt(n.ts)
lines(lags, acf.est + CI.AR, col = "red", lty = 2)
lines(lags, acf.est - CI.AR, col = "red", lty = 2)
# Add the usual IID 95% confidence bounds, ±1.96/sqrt(n), for comparison
CI.IID <- rep(1.96 / sqrt(n.ts), n.lags)
lines(lags, -CI.IID, col = "gray", lty = 2)
lines(lags, CI.IID, col = "gray", lty = 2)
# Add the zero-reference line and legend
abline(h = 0, lty = "dashed")
legend("bottomleft",
     legend = c("IID 95% CI", "AR(1) 95% CI", "AR(1) model"),
     lty = c(2, 2, NA),
     pch = c(NA, NA, 1),
     col = c("gray", "red", "red"),
     bty = "n")
}

# --- Set plotting parameters and display the ACF comparison -----
par(mgp = c(2, 0.7, 0), mar = c(3.5, 3.5, 1.4, 0.6), family = "serif")
plot.ACFbartlettAR(ws)

```



```
# --- Alternatively, estimate the AR(1) model by maximum likelihood -----
# Fit an AR(1) model directly rather than estimating phi from the lag-1 ACF
phi_hat_mle <- arima(ws, order = c(1, 0, 0))
phi_hat_mle
```

```
##
## Call:
## arima(x = ws, order = c(1, 0, 0))
##
## Coefficients:
##      ar1  intercept
##      0.906   -1.1136
## s.e.  0.037    0.6035
##
## sigma^2 estimated as 0.4615:  log likelihood = -132.99,  aic = 271.99
```

```
# --- Center the series if a mean-zero representation is needed -----
# Subtract the sample mean so that the transformed series has sample mean zero
ws.centered <- ws - mean(ws)
```

One-step-ahead forecast

```
# -- One-step-ahead predictions and errors # --- One-step-ahead prediction using the fitted AR(1) model
```

```

# For  $t = 2, \dots, n$ , predict  $x_t$  using the previous observation  $x_{t-1}$ 
# The mean-adjusted AR(1) predictor is:
#  $\hat{x}_t = \bar{x} + \hat{\phi} (x_{t-1} - \bar{x})$ 
ws.hat <- phi.ws * ws.centered[1:(n - 1)] + xbar_ws

# Compute one-step-ahead prediction errors:
#  $e_t = \hat{x}_t - x_t$ 
zt.ws <- ws.hat - ws[2:n]

# --- Plot the observed series and one-step-ahead predictions -----
par(las = 1, mgp = c(2, 0.7, 0), mar = c(3.5, 3.5, 1.2, 0.6),
    family = "serif")

# Plot the observed wind speed time series
plot(ws, col = "blue", xlab = "Time", type = "b",
     ylab = expression(x[t]),
     main = "One-Step-Ahead Prediction", cex = 0.75)

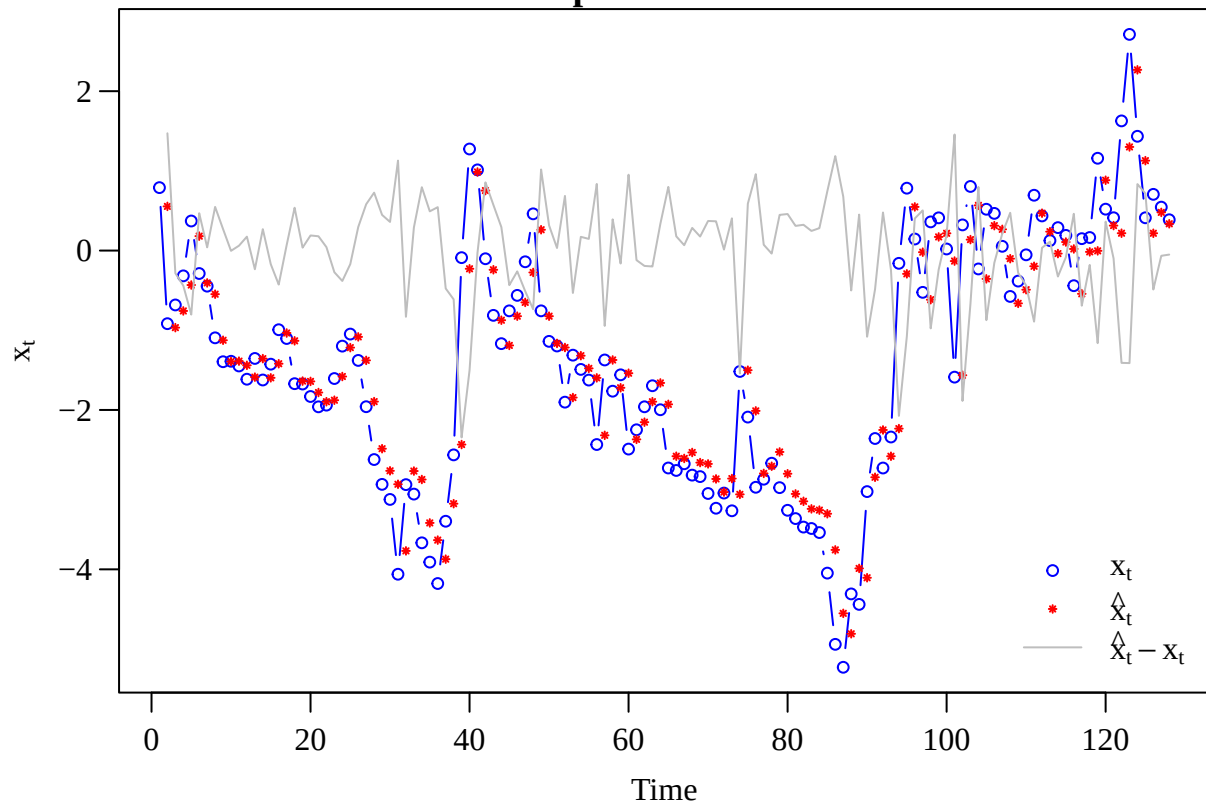
# Add the one-step-ahead predictions for  $t = 2, \dots, n$ 
points(2:n, ws.hat, pch = 8, col = "red", cex = 0.375)

# Add the corresponding one-step-ahead prediction errors
lines(2:n, zt.ws, col = "gray")

# Add a legend identifying the observations, predictions, and errors
legend("bottomright",
     legend = expression(x[t], hat(x)[t], hat(x)[t] - x[t]),
     col = c("blue", "red", "gray"),
     pch = c(1, 8, NA), lty = c(NA, NA, "solid"),
     pt.cex = c(0.75, 0.375, 1), inset = 0.01, bty = "n")

```

One-Step-Ahead Prediction



```
# --- Compare prediction-error variance with the variance of the series -----
```

```
# A useful AR(1) model should reduce uncertainty relative to predicting
# without using the previous observation; compare the prediction-error
# variance with the marginal variance of the original series
```

```
cat("\nQuick AR(1) estimate (lag-1 ACF): phi_hat =", round(phi.ws, 4), "\n")
```

```
##
```

```
## Quick AR(1) estimate (lag-1 ACF): phi_hat = 0.8911
```

```
cat("Var(prediction errors) =", round(var(zt.ws), 4), "\n")
```

```
## Var(prediction errors) = 0.4629
```

```
cat("Var(series)          =", round(var(ws), 4), "\n")
```

```
## Var(series)          = 2.5025
```

Fill in missing value example

Simulate an AR(-0.9)

```

# Generate 100 observations from the AR(1) model:
#  $X_t = \phi * X_{t-1} + Z_t$ ,  $Z_t \sim N(0, 1)$ 
generate.AR1.ts <- function(phi = 0.0){
  ts <- rep(0, 100)
  # Initialize  $X_1$  from the stationary distribution
  # For innovation variance 1,  $\text{Var}(X_t) = 1 / (1 - \phi^2)$ 
  ts[1] <- rnorm(1) / sqrt(1 - phi^2)
  # Generate subsequent observations recursively from the AR(1) model
  for(i in 2:100) ts[i] <- phi * ts[i - 1] + rnorm(1)
  ts
}

# Set the random seed for reproducibility
set.seed(123)

# Simulate an AR(1) process with  $\phi = -0.9$ 
# A negative  $\phi$  produces an oscillating pattern between successive observations
ar1.ts <- generate.AR1.ts(-0.9)

# Plot the simulated AR(1) time series

# Load scales to add transparency to the connecting line
library(scales)

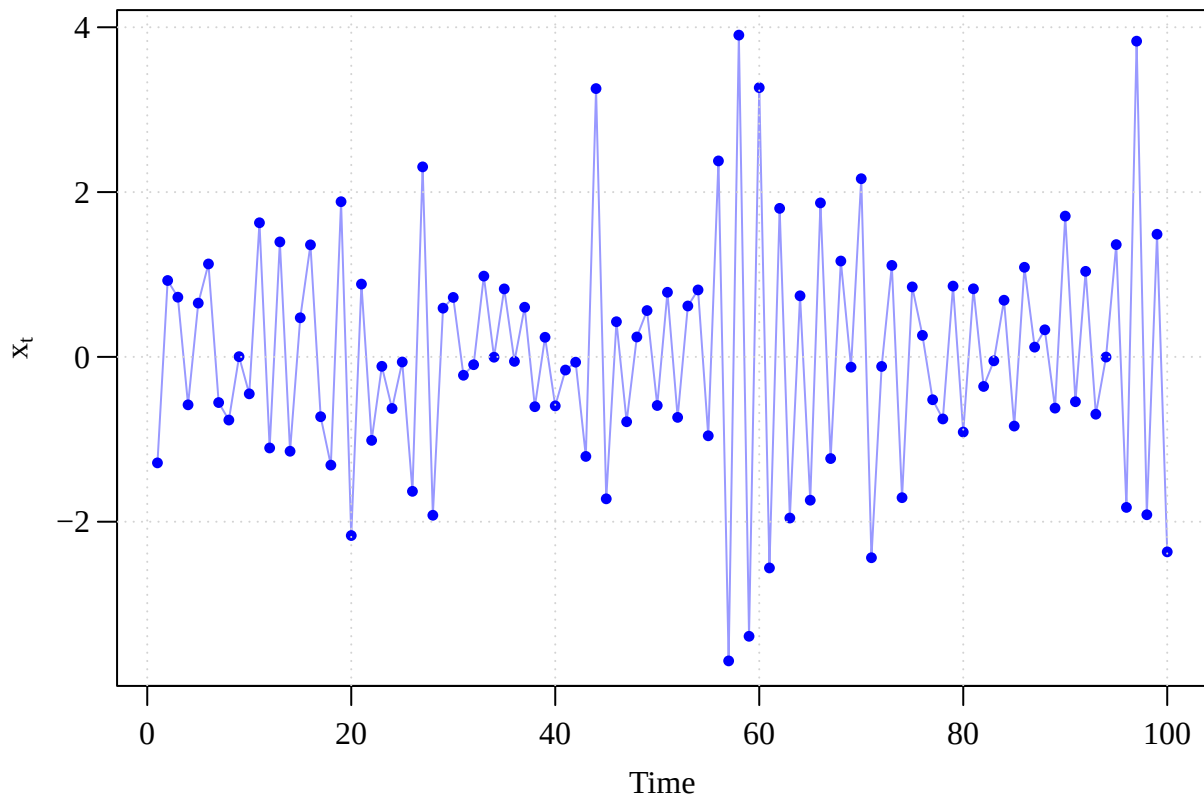
# Set graphical parameters
par(las = 1, mgp = c(2, 0.7, 0), mar = c(3.5, 3.5, 1.4, 0.6),
    family = "serif")

# Plot the time series with a semi-transparent line
plot(ar1.ts, col = alpha("blue", 0.4), xlab = "Time", type = "l",
     ylab = expression(x[t]), cex = 0.5)

# Add points to highlight individual observations
points(ar1.ts, pch = 16, cex = 0.75, col = "blue")

# Add grid lines for easier visual comparison across time
grid()

```



Let's remove some data to illustrate how to fill in missing values using forecasting algorithm

```
# Subsample the AR(1) series by retaining every other observation

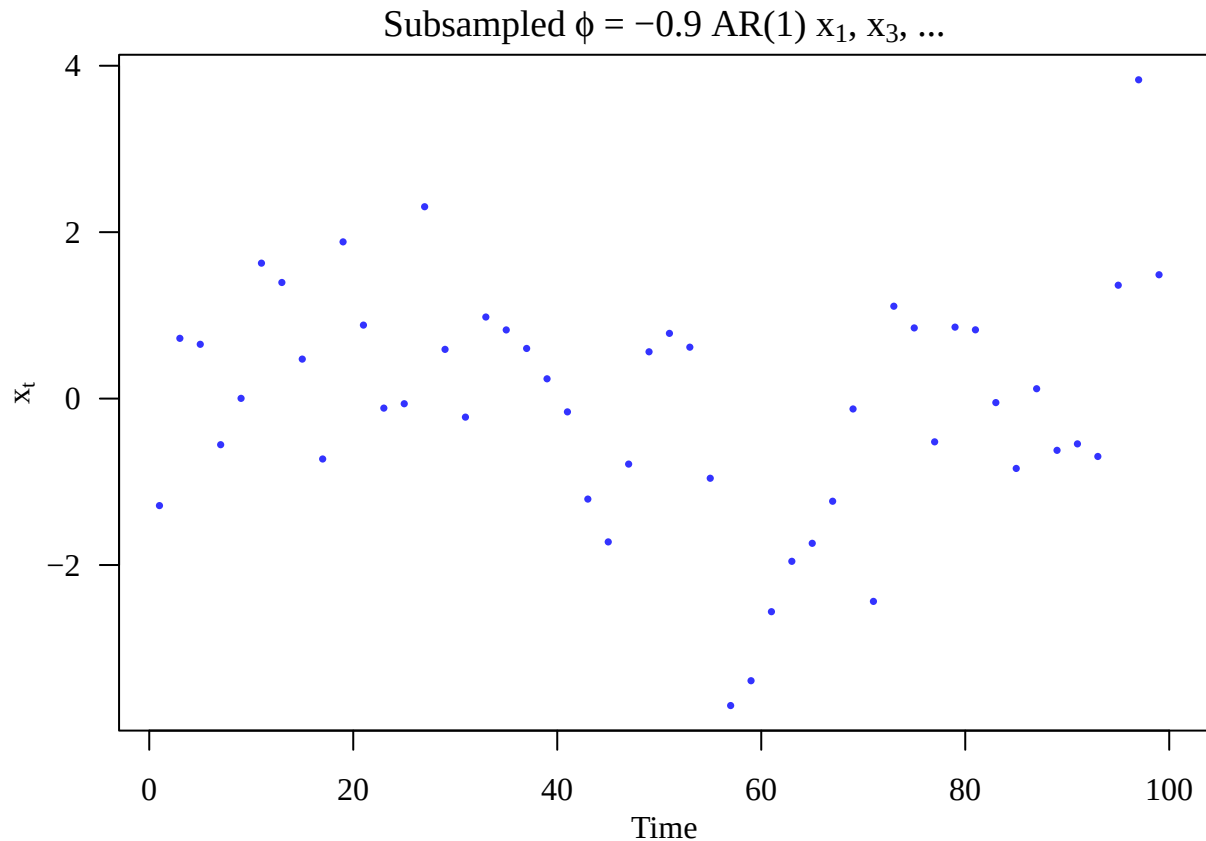
# Create a copy of the original AR(1) series
ar1.ts.subsampled <- ar1.ts

# Remove observations at even time points, leaving X_1, X_3, X_5, ...
# The retained observations are therefore separated by two time steps
ar1.ts.subsampled[seq(2, 100, 2)] <- NA

# Plot the subsampled AR(1) time series

# Set graphical parameters
par(las = 1, mgp = c(2, 1, 0), mar = c(3.5, 3.5, 1.4, 0.6),
    family = "serif")

# Plot the retained observations X_1, X_3, X_5, ...
# For an AR(1) process, Corr(X_t, X_{t-2}) = phi^2;
# thus subsampling changes the dependence between consecutive retained values
plot(ar1.ts.subsampled, xlab = "Time", type = "b",
     ylab = expression(x[t]),
     main = expression(paste("Subsampled ", phi, " = -0.9 AR(1) ",
                             x[1], " ", x[3], " ...")),
     cex = 0.5, col = alpha("blue", 0.8), pch = 16)
```



Fill in “missing” values

$$\hat{X}_2 = \phi(X_1 + X_3)/(1 + \phi^2)$$

$$\text{MSPE} = \frac{\sigma^2}{1 + \phi^2}$$

```
# Copy the original AR(1) series for prediction
ar1.ts.predicted <- ar1.ts

# Predict the missing even-indexed observations using their two neighbors
# Conditional expectation for an AR(1) process with phi = -0.9
ar1.ts.predicted[seq(2, 98, 2)] <-
  -0.9 * (ar1.ts[seq(1, 97, 2)] + ar1.ts[seq(3, 99, 2)]) / 1.81

# No two-sided prediction is available for the last observation
ar1.ts.predicted[100] <- NA

# Set plotting parameters
par(las = 1, mgp = c(2, 1, 0), mar = c(3.5, 3.5, 1.4, 0.6),
    family = "serif")

# Plot the series with predicted values filled in
plot(ar1.ts.predicted, col = alpha("blue", 0.4), xlab = "Time", type = "l",
     ylab = expression(x[t]), cex = 0.5)

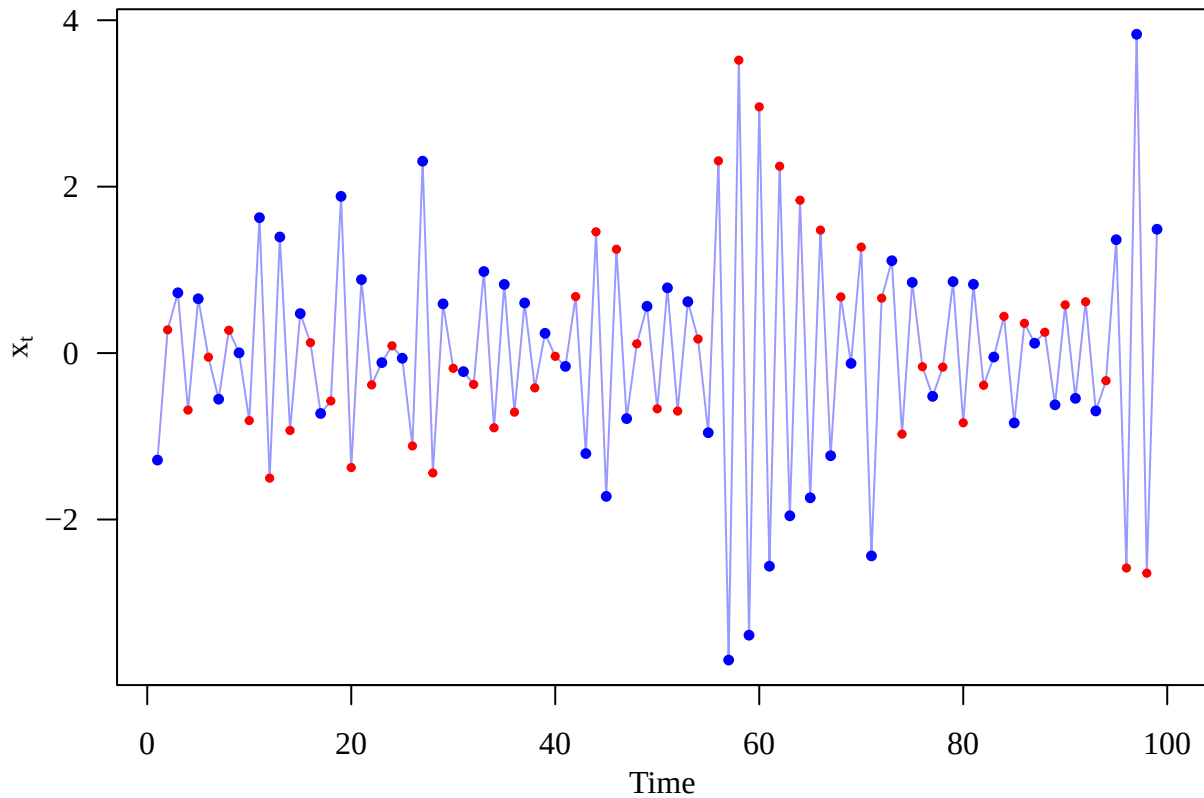
# Highlight predicted observations in red
```

```

xs <- seq(2, 98, 2)
points(xs, ar1.ts.predicted[xs], pch = 19, col = "red", cex = 0.5)

# Highlight observed observations in blue
xo <- seq(1, 99, 2)
points(xo, ar1.ts.subsampled[xo], col = "blue", cex = 0.75, pch = 16)

```



Prediction Errors from Best Linear Predictor

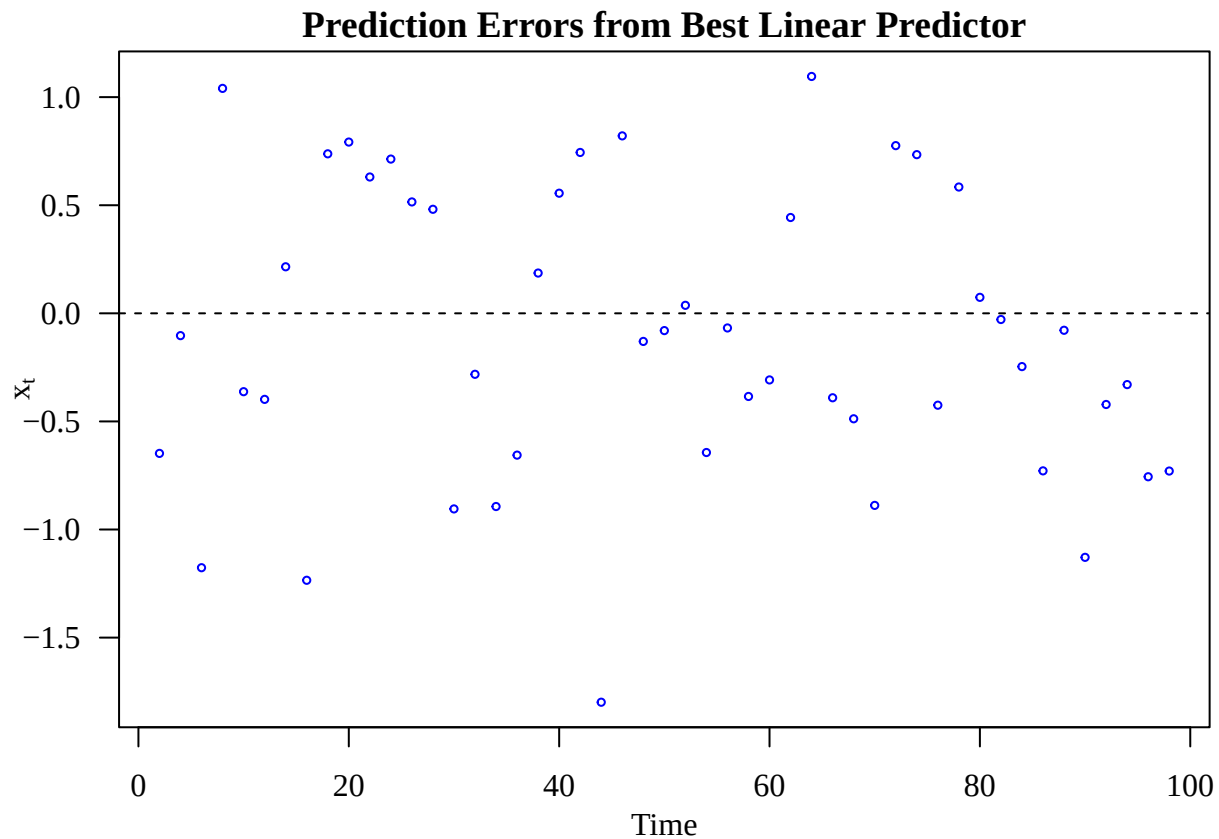
```

# Set plotting parameters
par(las = 1, mgp = c(2, 1, 0), mar = c(3.5, 3.5, 1.4, 0.6),
    family = "serif")

# Plot prediction errors at the missing even-indexed time points
plot(xs, (ar1.ts.predicted - ar1.ts)[xs], col = "blue",
     xlab = "Time", type = "p", ylab = expression(x[t]),
     main = "Prediction Errors from Best Linear Predictor", cex = 0.5)

# Add a reference line at zero prediction error
abline(h = 0, lty = 2)

```



Ireland wind data case study

Load and plot the data

In this case study, we use the data at the Rosslare station from 1965 to 1969.

```
# Load the gstat package and wind dataset
library(gstat)
data(wind)

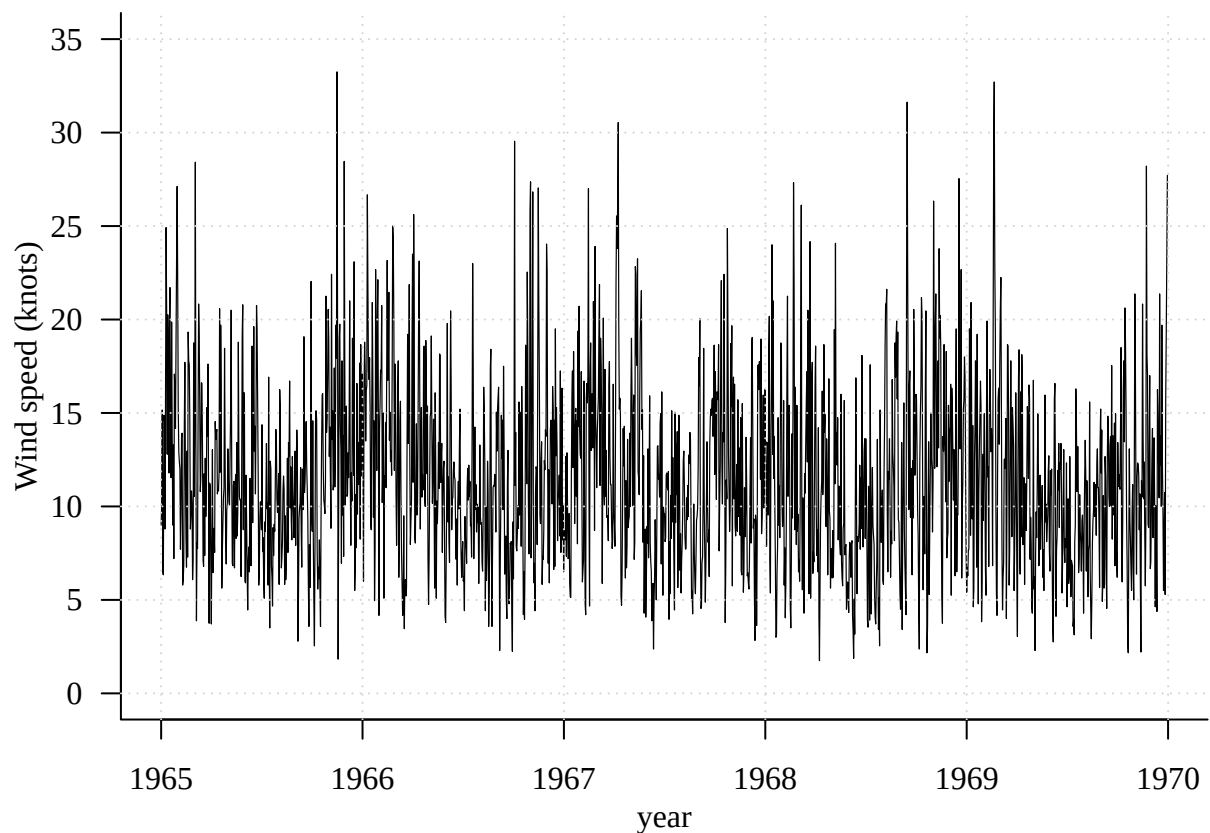
# Extract Rosslare wind speeds for 1965--1969
id <- which(wind$year %in% 65:69)
rosslare <- wind$ROS[id]

# Create a continuous time variable in years
year <- seq(from = 1965, by = 1 / 365.25, length = length(rosslare))

# Set plotting parameters
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), mgp = c(2, 1, 0), las = 1,
    family = "serif")

# Plot the daily wind speed time series
plot(year, rosslare, type = "l", ylim = c(0, 35), lwd = 0.6,
     xlab = "year", ylab = "Wind speed (knots)")
```

```
# Add grid lines
grid()
```



Deseasonalization: Harmonic Regression

We use harmonic regression with 4 harmonics per year to model the seasonal components.

```
# Create harmonic terms for modeling seasonal variation
Harmonic <- function(year, K){
  t <- outer(2 * pi * year, 1:K)
  return(cbind(apply(t, 2, cos), apply(t, 2, sin)))
}

# Use the first four harmonics
harmonics <- Harmonic(year, 4)

# Fit a harmonic regression model for the seasonal pattern
harm.model <- lm(rosslare ~ harmonics)
summary(harm.model)

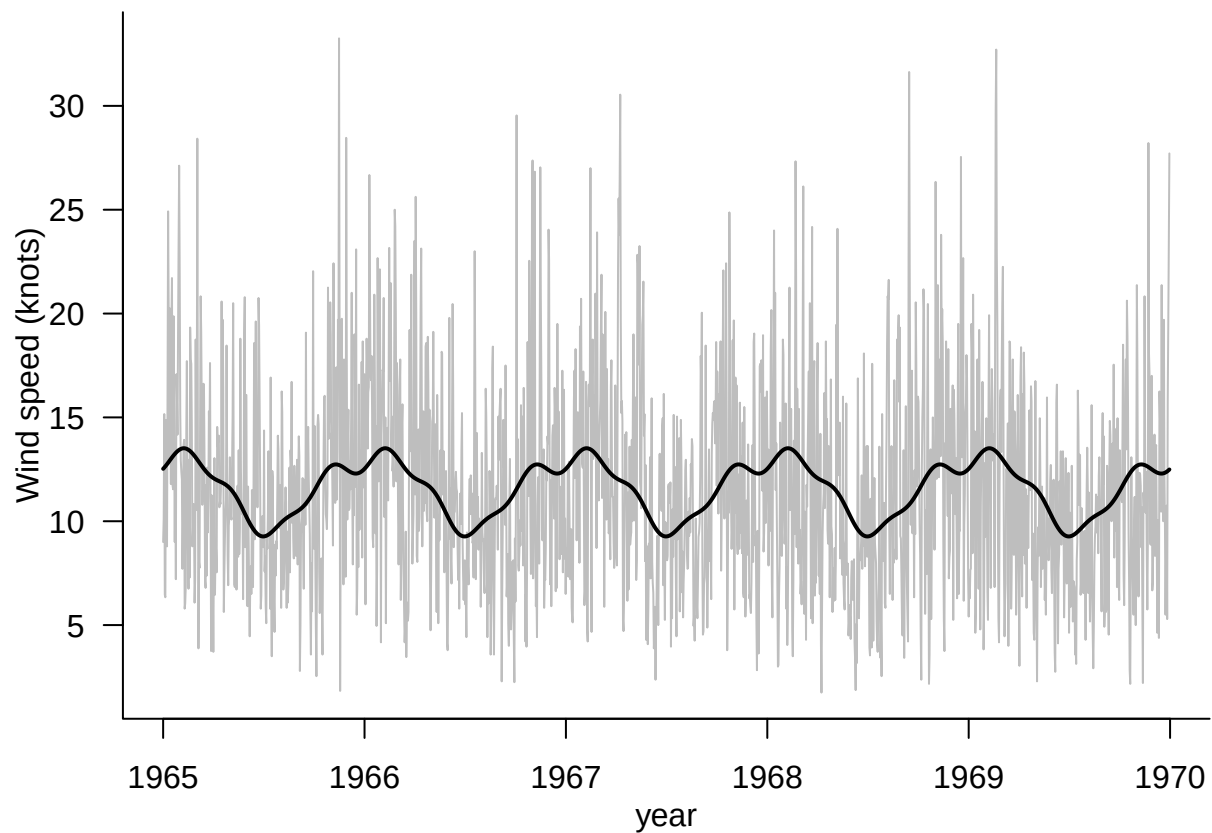
##
## Call:
## lm(formula = rosslare ~ harmonics)
##
## Residuals:
```

```
##      Min      1Q   Median      3Q      Max
## -10.8538 -3.3813 -0.4892   2.8395  20.8290
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 11.584141   0.112377 103.083 < 2e-16 ***
## harmonics1   1.687468   0.158936  10.617 < 2e-16 ***
## harmonics2  -0.435273   0.158936  -2.739 0.00623 **
## harmonics3  -0.060047   0.158936  -0.378 0.70562
## harmonics4  -0.251396   0.158936  -1.582 0.11388
## harmonics5   0.412363   0.158915   2.595 0.00954 **
## harmonics6   0.003874   0.158915   0.024 0.98055
## harmonics7   0.107245   0.158915   0.675 0.49985
## harmonics8   0.217870   0.158915   1.371 0.17055
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.802 on 1817 degrees of freedom
## Multiple R-squared:  0.06771,    Adjusted R-squared:  0.06361
## F-statistic: 16.5 on 8 and 1817 DF,  p-value: < 2.2e-16
```

```
# Set plotting parameters
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), mgp = c(2, 1, 0), las = 1)

# Plot the observed wind speeds
plot(year, rosslare, type = "l",
      xlab = "year", ylab = "Wind speed (knots)", col = "grey")

# Add the fitted seasonal pattern from the harmonic regression
lines(year, fitted(harm.model), lwd = 2)
```



ACF Plots: Original and Deseasonalized Series

Let's plot the ACF and PACF plots to investigate the possible order for the ARMA model.

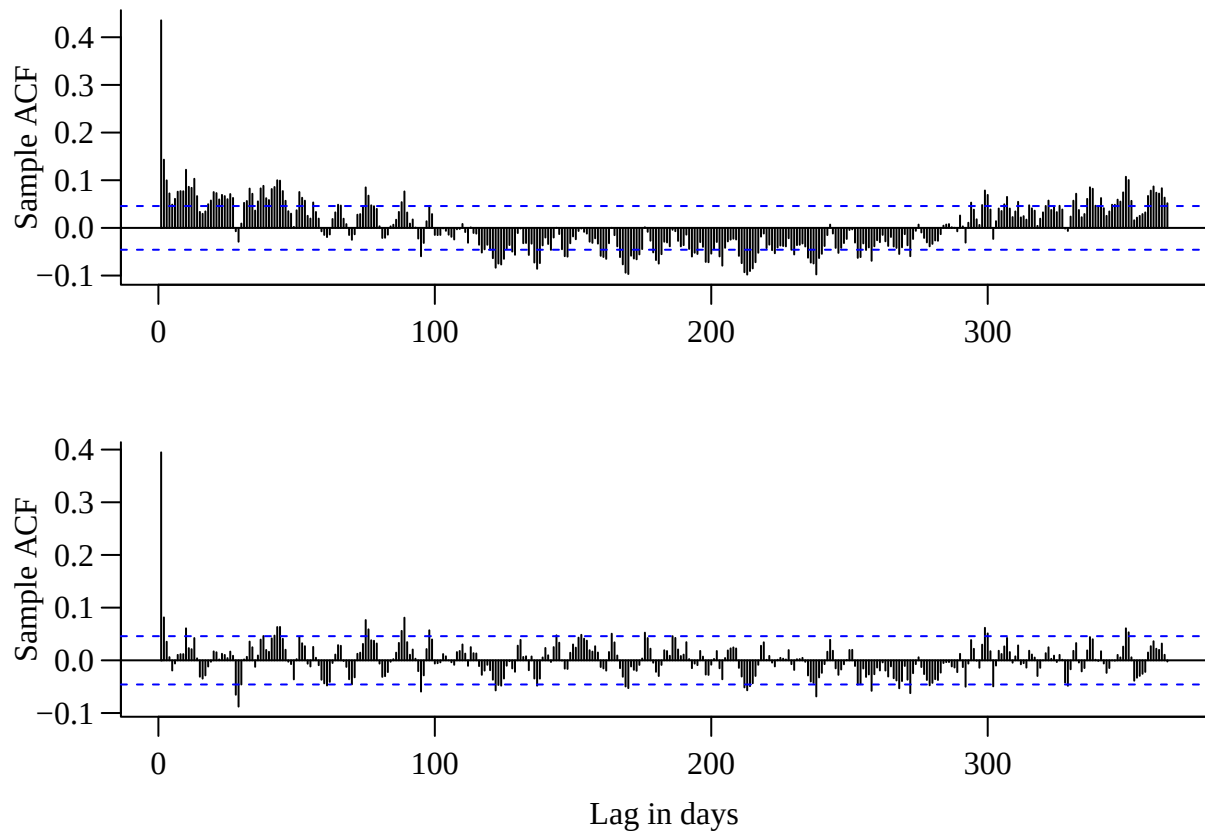
```
# Load the forecast package for ACF plots
library(forecast)

## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo

# Display the ACFs in two panels
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), mgp = c(2, 0.7, 0), las = 1,
    mfrow = c(2, 1), family = "serif")

# Sample ACF of the original wind speed series
Acf(rosslare, lag.max = 365, xlab = "",
    ylab = "Sample ACF", main = "")

# Sample ACF after removing the fitted seasonal pattern
Acf(resid(harm.model), lag.max = 365, xlab = "Lag in days",
    ylab = "Sample ACF", main = "")
```



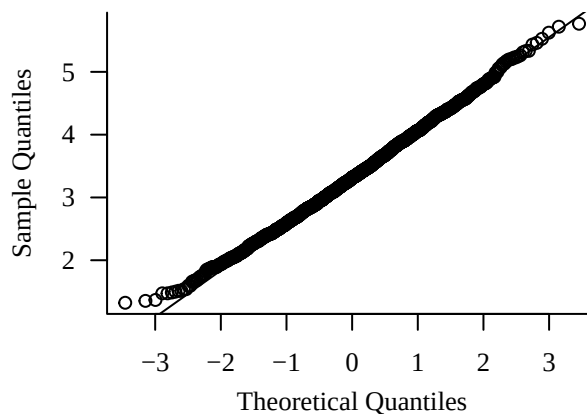
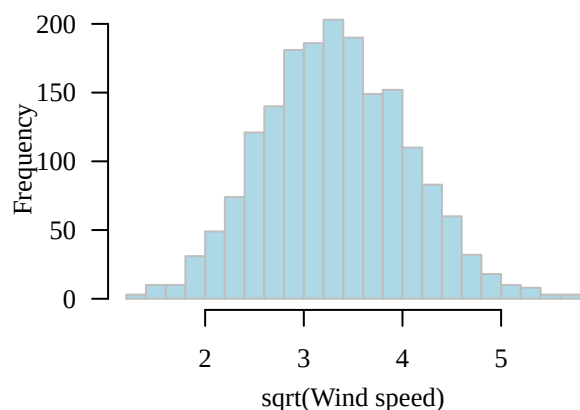
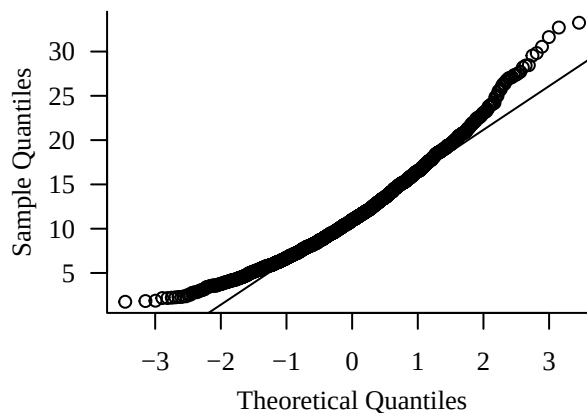
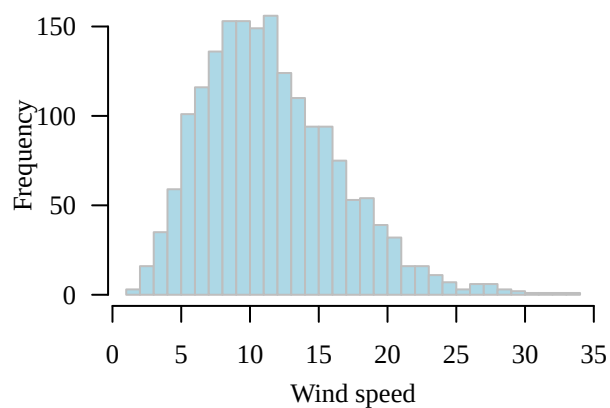
Apply transformation to make wind speed more Gaussian like

Now look at a histogram of the values, along with the normal quantile-quantile plot.

```
# Display distributional diagnostics in a 2 x 2 layout
par(mfrow = c(2, 2), bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1,
    mgp = c(2.2, 1, 0), family = "serif")

# Histogram and normal Q-Q plot of the original wind speeds
hist(rosslare, 40, main = "", xlab = "Wind speed",
     col = "lightblue", border = "gray")
qqnorm(rosslare, main = "")
qqline(rosslare)

# Histogram and normal Q-Q plot after square-root transformation
hist(sqrt(rosslare), 25, main = "", xlab = "sqrt(Wind speed)",
     col = "lightblue", border = "gray")
qqnorm(sqrt(rosslare), main = "")
qqline(sqrt(rosslare))
```



Now take square roots of the original data and deseasonalizeagain!

```
# Apply the square-root transformation to the wind speed series
sqrt.rosslare <- sqrt(rosslare)
```

```
# Refit the seasonal harmonic model without an intercept
harm.model <- lm(sqrt.rosslare ~ harmonics[, 1:4] - 1)
summary(harm.model)
```

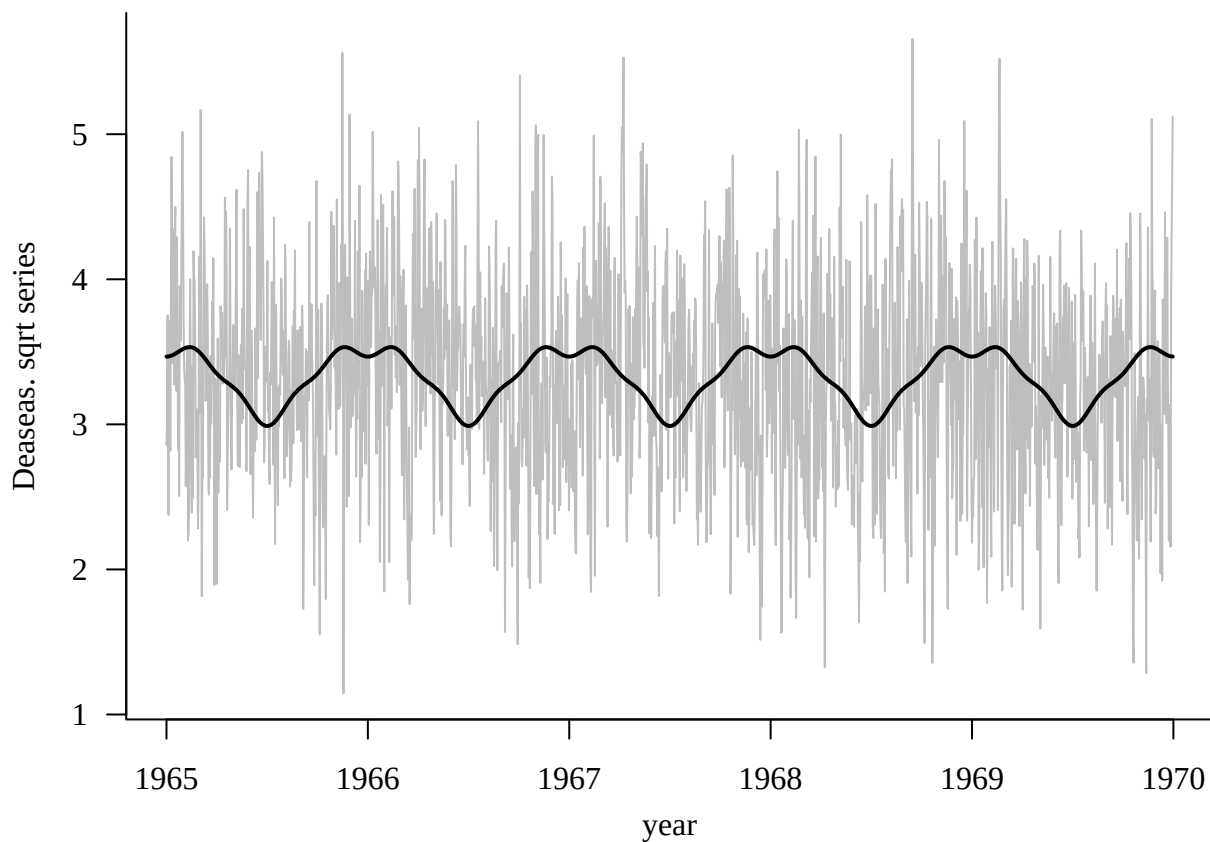
```
##
## Call:
## lm(formula = sqrt.rosslare ~ harmonics[, 1:4] - 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
##  1.146   2.848   3.316   3.799   5.656
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## harmonics[, 1:4]1  0.2391111  0.1126203   2.123  0.0339 *
## harmonics[, 1:4]2 -0.0606520  0.1126203  -0.539  0.5903
## harmonics[, 1:4]3 -0.0001588  0.1126203  -0.001  0.9989
## harmonics[, 1:4]4 -0.0363877  0.1126202  -0.323  0.7467
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.403 on 1822 degrees of freedom
## Multiple R-squared:  0.002684,    Adjusted R-squared:  0.0004944
## F-statistic: 1.226 on 4 and 1822 DF,  p-value: 0.2978
```

```
# Obtain the deseasonalized square-root series
sqrt.rosslare.ds <- resid(harm.model)

# Plot the deseasonalized square-root wind speed series
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.2, 1, 0),
    family = "serif")
plot(year, sqrt.rosslare.ds, type = "l", col = "gray",
     xlab = "year", ylab = "Deaseas. sqrt series")

# Add the fitted seasonal component
lines(year, fitted(harm.model) + mean(sqrt.rosslare.ds), lwd = 2)
```



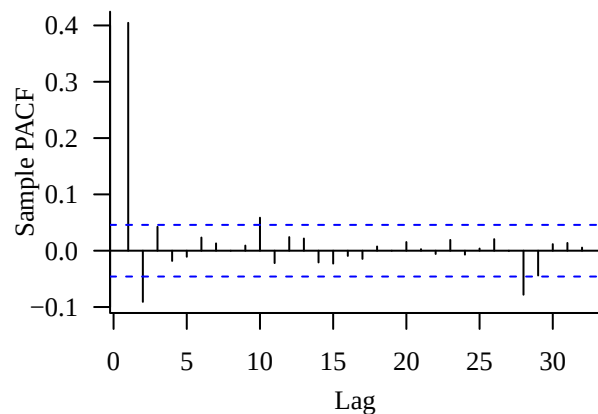
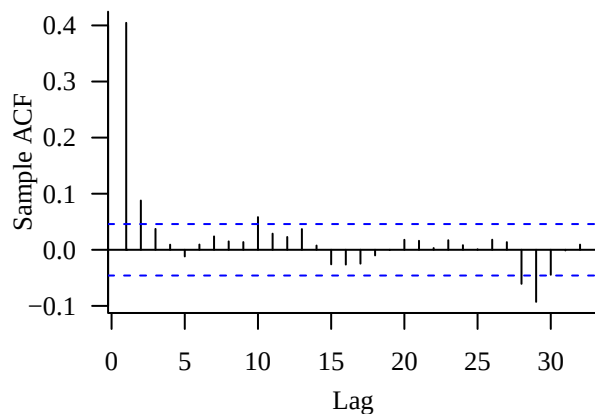
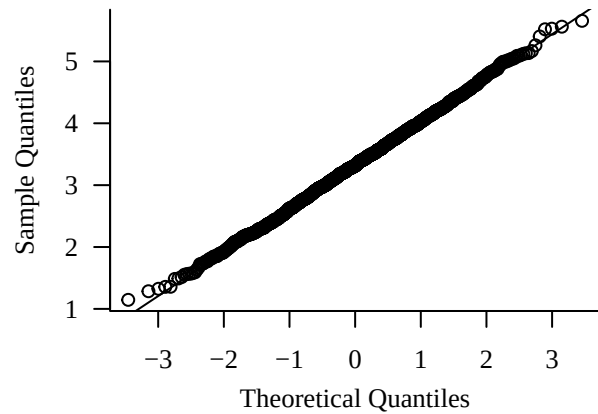
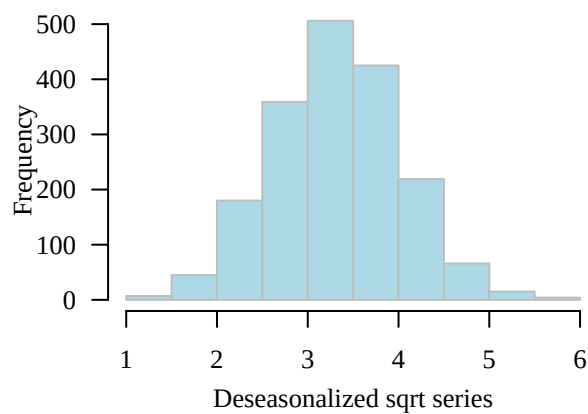
Checking Normality ACF/PACF

```
# Check the distribution of the deseasonalized series
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.2, 1, 0),
    mfrow = c(2, 2), family = "serif")

# Histogram and normal Q-Q plot
```

```
hist(sqrt.rosslare.ds, main = "", xlab = "Deseasonalized sqrt series",
     col = "lightblue", border = "gray")
qqnorm(sqrt.rosslare.ds, main = "")
qqline(sqrt.rosslare.ds)

# Examine the sample ACF and PACF
Acf(sqrt.rosslare.ds, main = "", ylab = "Sample ACF")
Acf(sqrt.rosslare.ds, main = "", type = "partial", ylab = "Sample PACF")
```



Model identification, fitting, and selection

Let's first fit an AR(1) Fit an AR(1) model

```
ar1.model <- arima(sqrt.rosslare.ds, order = c(1, 0, 0))
```

Summarize the fitted model

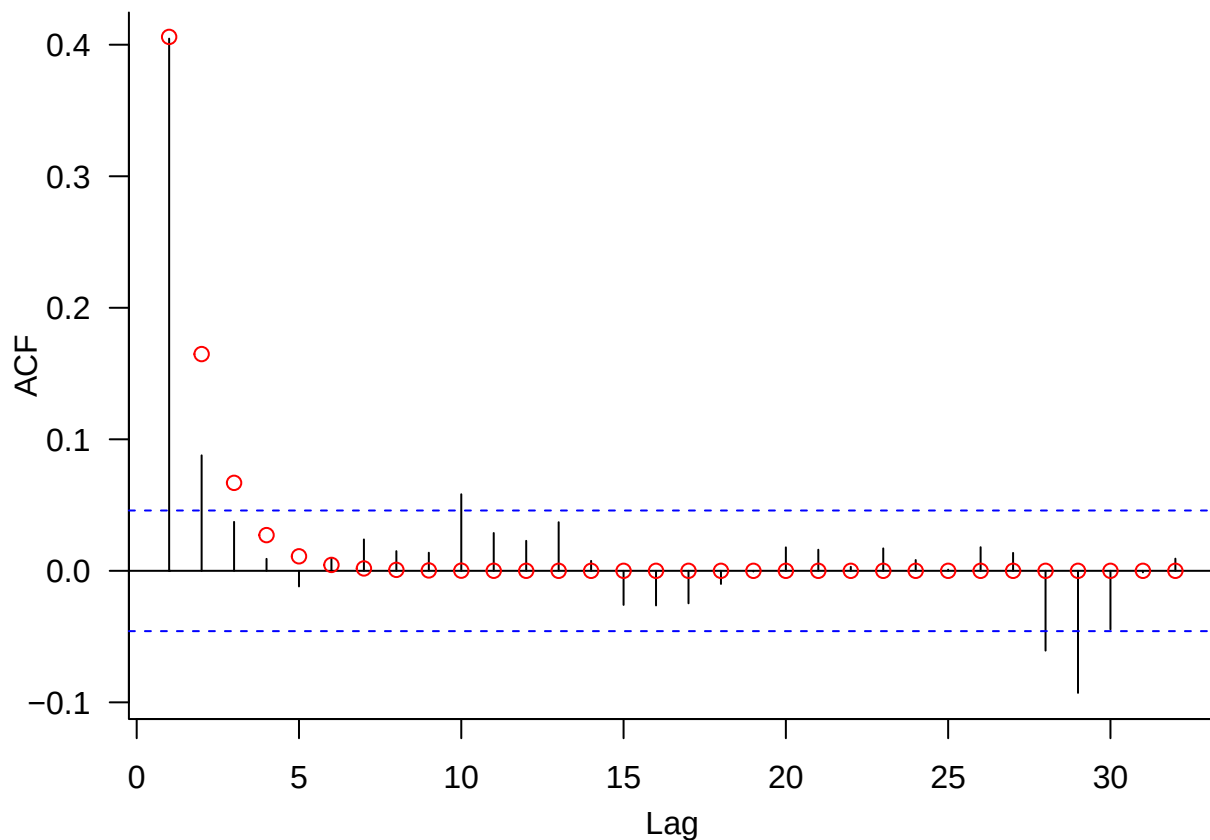
```
ar1.model
```

```
##
## Call:
## arima(x = sqrt.rosslare.ds, order = c(1, 0, 0))
##
## Coefficients:
```

```
##          ar1  intercept
##      0.4060      3.3257
## s.e.  0.0214      0.0254
##
## sigma^2 estimated as 0.4148:  log likelihood = -1787.72,  aic = 3581.43
```

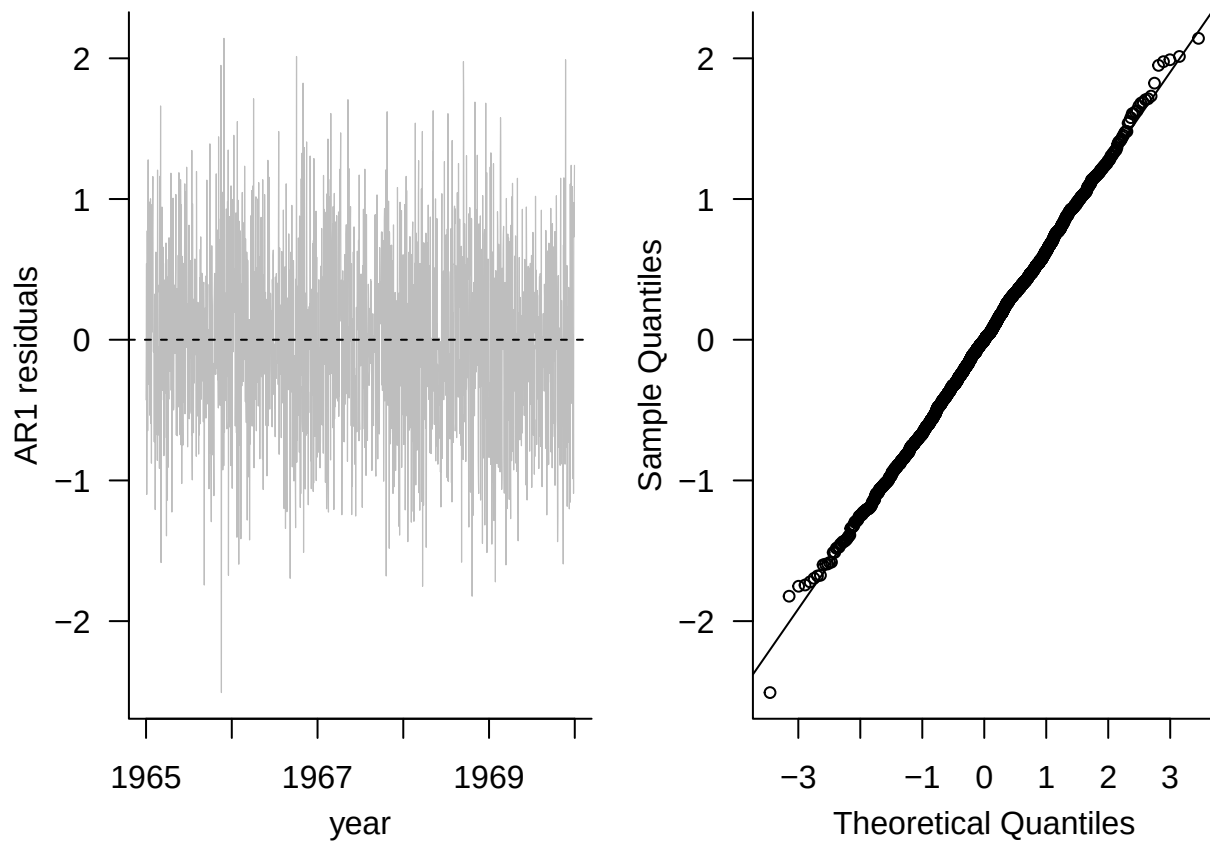
Sample and fitted ACF

```
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.2, 1, 0))
Acf(sqrt.rosslare.ds, main = "")
acf_true <- ARMAacf(ar = c(ar1.model$coef[1]), lag.max = 32)[-1]
points(1:32, acf_true, col = "red")
```



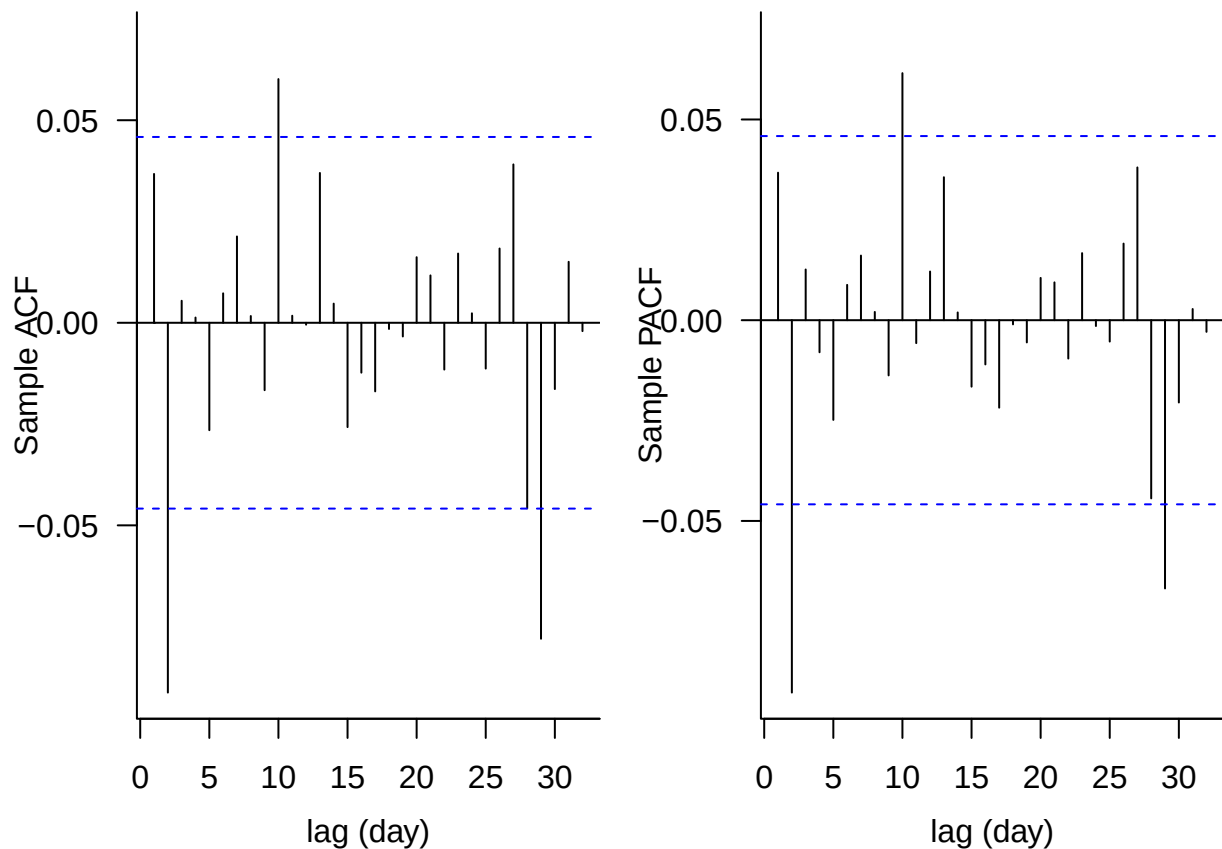
Extract residuals

```
ar1.resids <- resid(ar1.model)
## time series plot of the residuals
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.2, 1, 0),
    mfrow = c(1, 2))
plot(year, ar1.resids, type = "l", xlab = "year", ylab = "AR1 residuals",
     lwd = 0.6, col = "gray")
abline(h = 0, lty = 2)
## Normal Q-Q plot for the residuals
qqnorm(ar1.resids, main = "", cex = 0.75); qqline(ar1.resids)
```



Sample ACF and PACF of the residuals

```
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.4, 1, 0), mfrow = c(1, 2))
Acf(ar1.resids, ylab = "Sample ACF", xlab = "lag (day)", main = "")
Acf(ar1.resids, ylab = "Sample PACF", type = "partial", xlab = "lag (day)")
```



```
## Carry out the Ljung-Box test
Box.test(ar1.resids, lag = 32, type = "Ljung-Box", fitdf = 1)
```

```
##
## Box-Ljung test
##
## data: ar1.resids
## X-squared = 53.142, df = 31, p-value = 0.00794
```

```
(ar2.model <- arima(sqrt.rosslare.ds, order = c(2, 0, 0)))
```

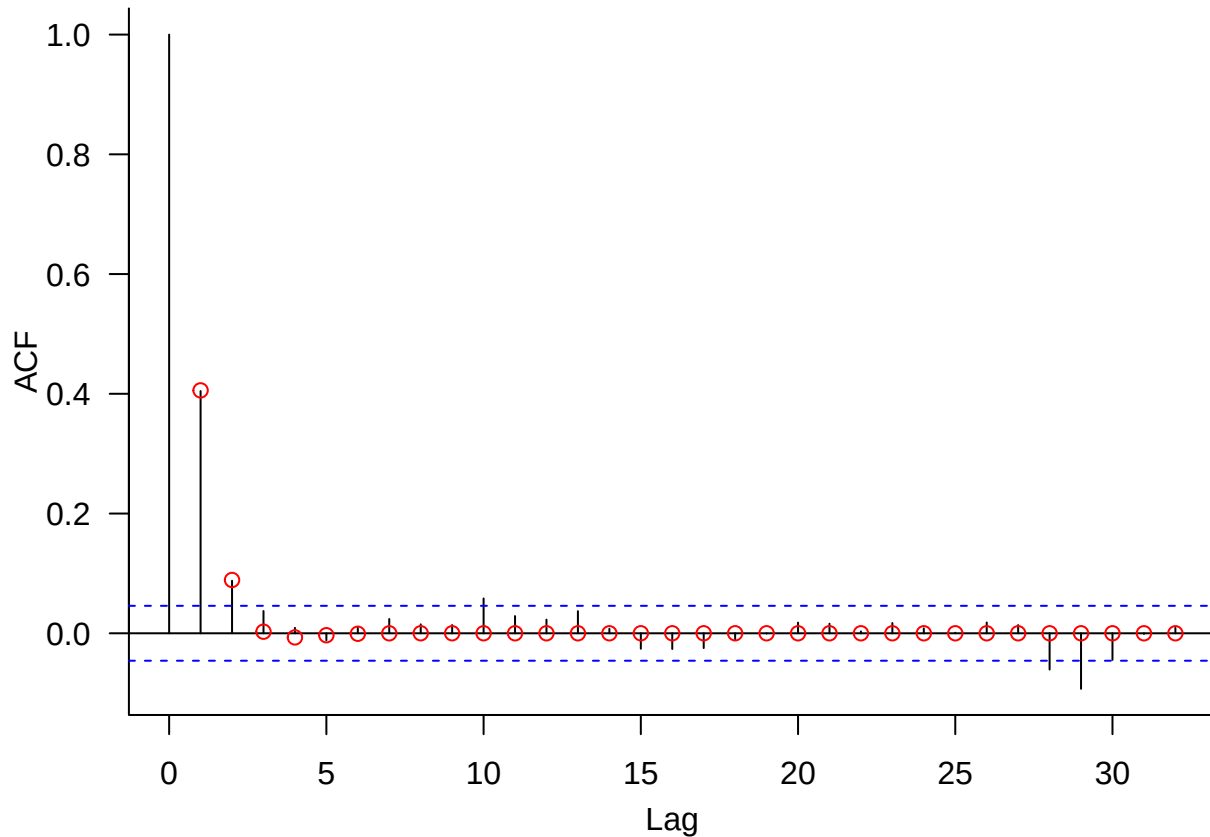
Fit an AR(2) model

```
##
## Call:
## arima(x = sqrt.rosslare.ds, order = c(2, 0, 0))
##
## Coefficients:
##          ar1          ar2  intercept
##          0.4425  -0.0905       3.3254
## s.e.  0.0233   0.0233    0.0232
##
## sigma^2 estimated as 0.4114:  log likelihood = -1780.23,  aic = 3568.46
```

```

par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.2, 1, 0))
acf(sqrt.rosslare.ds, main = "")
acf_true <- ARMAacf(ar = c(ar2.model$coef[1:2]), lag.max = 32)[-1]
points(1:32, acf_true, col = "red")

```



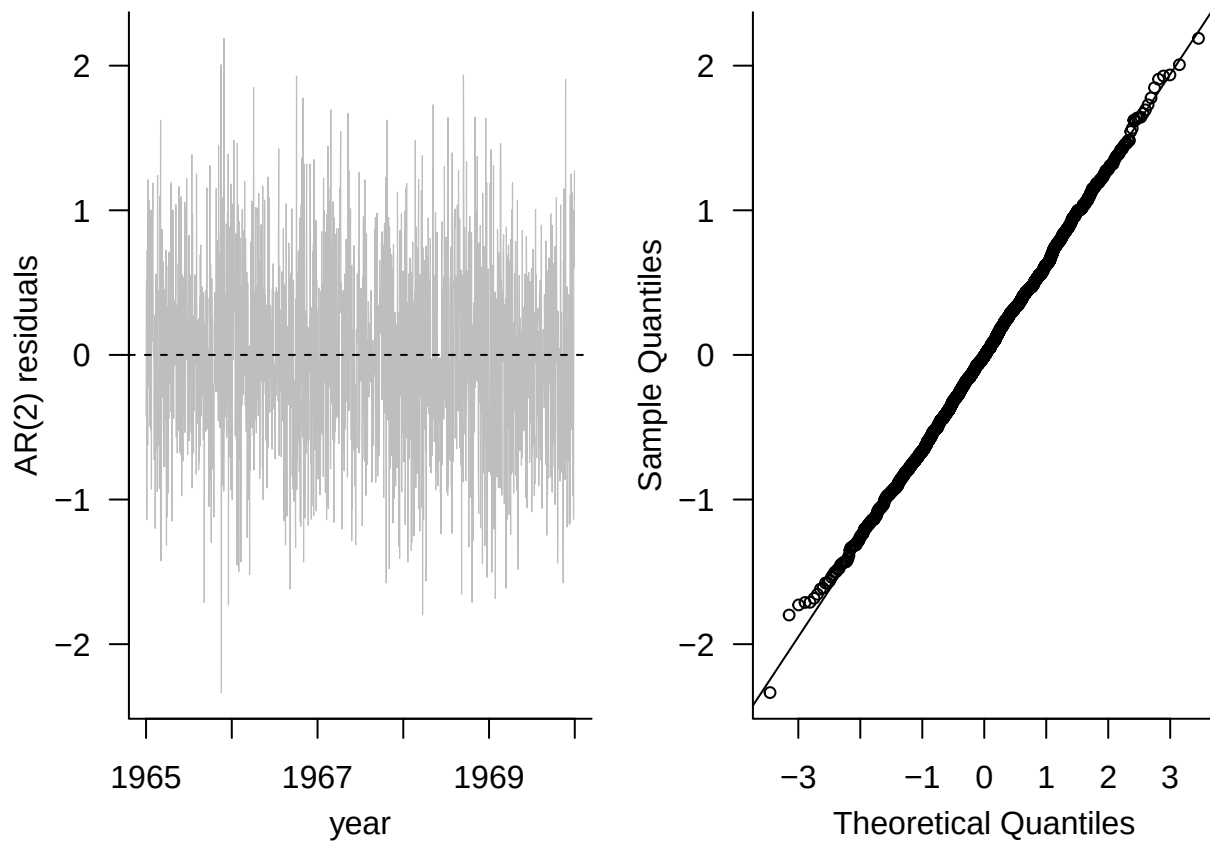
```

## extract the residuals
ar2.resids <- resid(ar2.model)

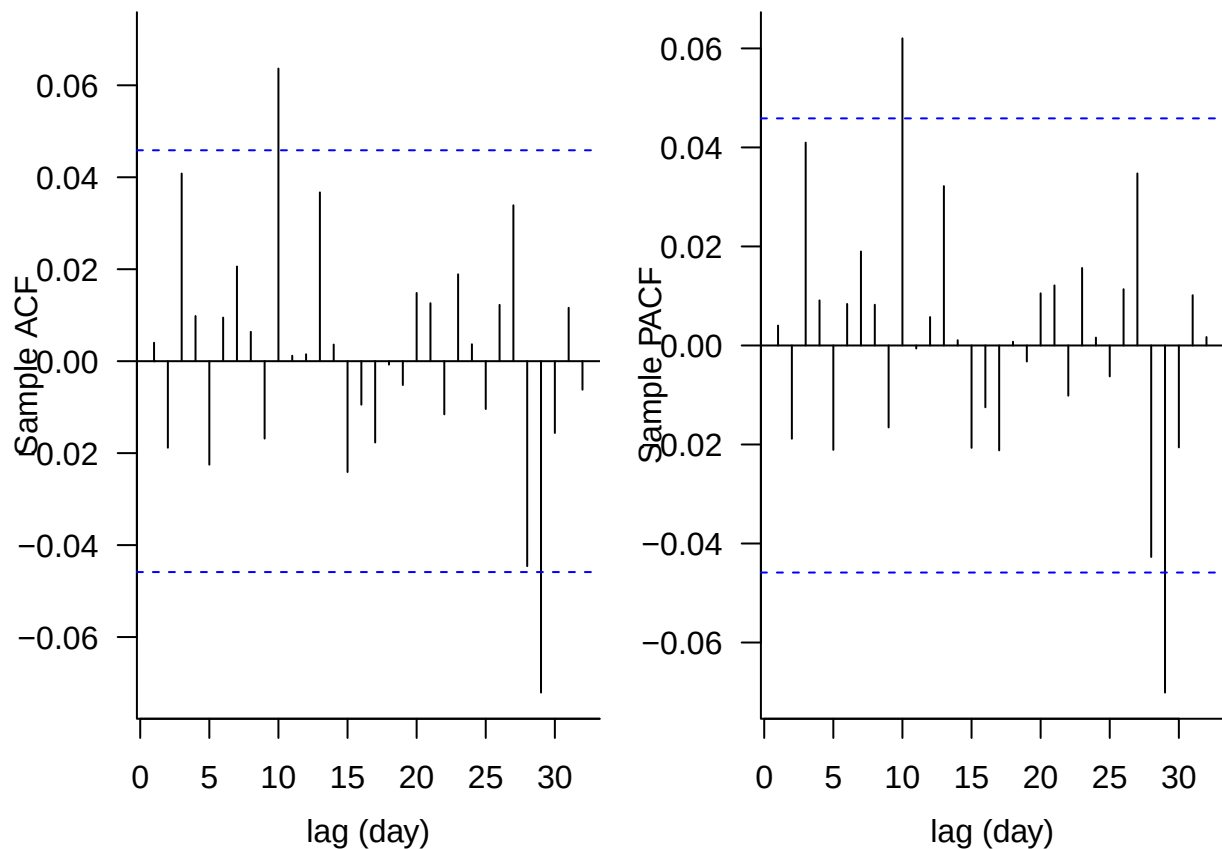
## time series plot of the residuals
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.2, 1, 0), mfrow = c(1, 2))
plot(year, ar2.resids, type = "l", xlab = "year",
      ylab = "AR(2) residuals", lwd = 0.6, col = "gray")
abline(h = 0, lty = 2)

## Normal Q-Q plot for the residuals
qqnorm(ar2.resids, main = "", cex = 0.75); qqline(ar2.resids)

```



```
## Sample ACF and PACF of the residuals
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.4, 1, 0), mfrow = c(1, 2))
Acf(ar2.resids, ylab = "Sample ACF", xlab = "lag (day)", main = "")
pacf(ar2.resids, ylab = "Sample PACF", xlab = "lag (day)")
```



```
## Carry out the Ljung-Box test
Box.test(ar2.resids, lag = 32, type = "Ljung-Box", fitdf = 2)
```

```
##
## Box-Ljung test
##
## data: ar2.resids
## X-squared = 36.548, df = 30, p-value = 0.1907
```

```
(arma11.model <- arima(sqrt.rosslare.ds, order = c(1, 0, 1)))
```

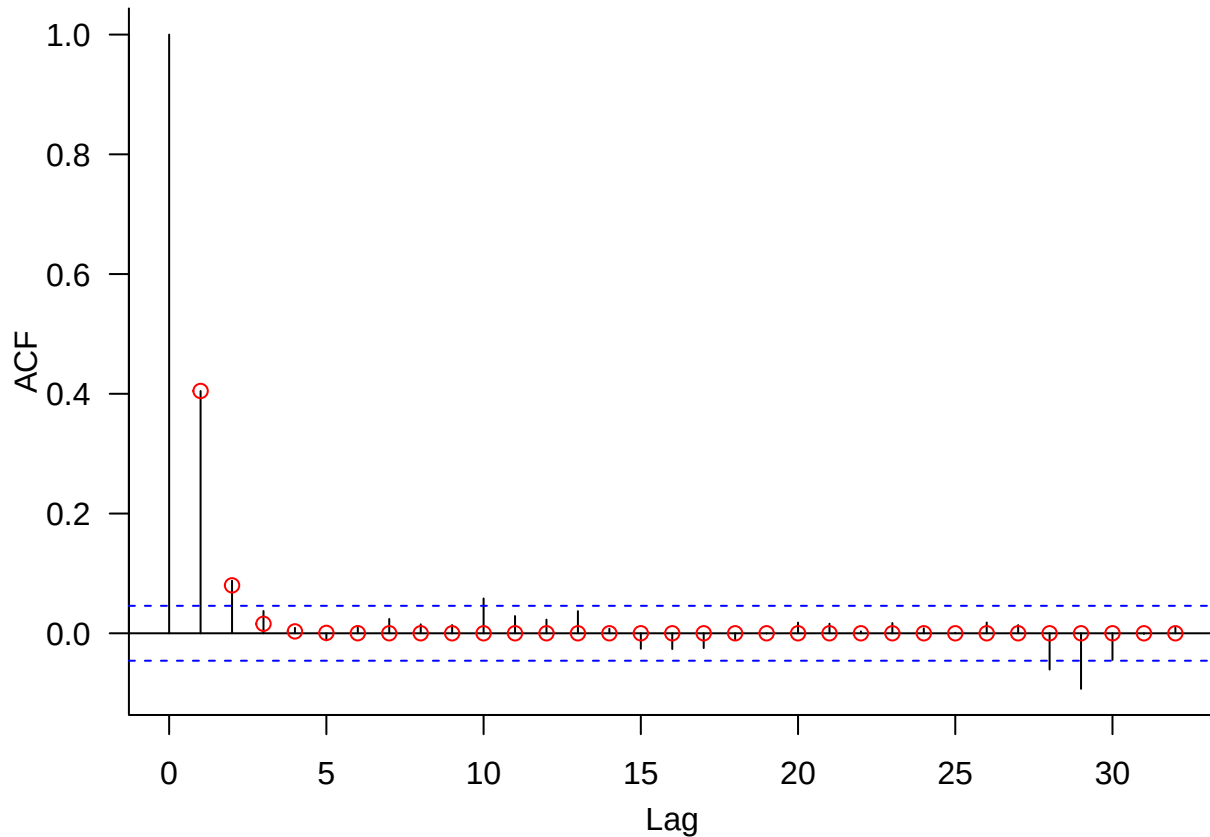
Fit an ARMA(1,1) model

```
##
## Call:
## arima(x = sqrt.rosslare.ds, order = c(1, 0, 1))
##
## Coefficients:
##          ar1          ma1  intercept
##       0.1978  0.2502      3.3254
## s.e.  0.0556  0.0553      0.0234
##
## sigma^2 estimated as 0.4108:  log likelihood = -1778.82,  aic = 3565.64
```

```

par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.2, 1, 0))
acf(sqrt.rosslare.ds, main = "")
acf_true <- ARMAacf(ar = arma11.model$coef[1], ma = arma11.model$coef[2], lag.max = 32)[-1]
points(1:32, acf_true, col = "red")

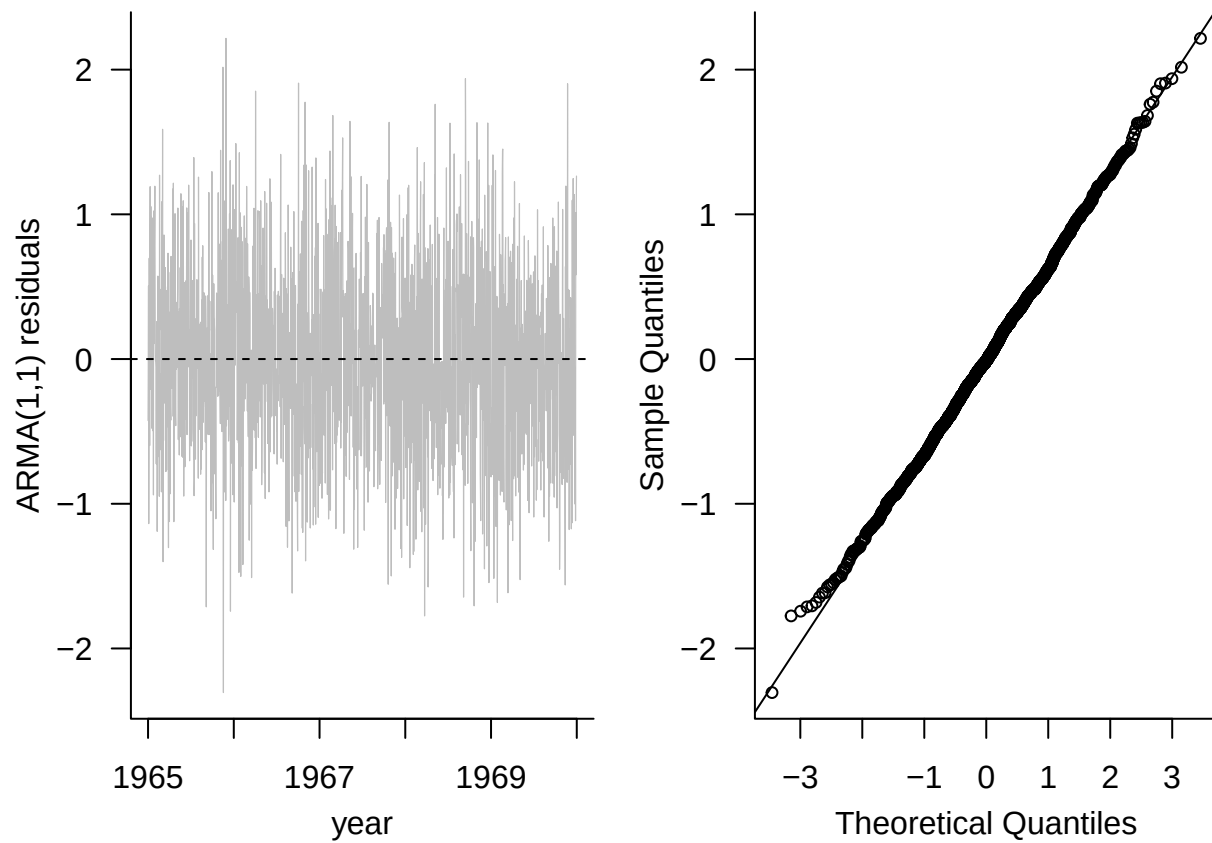
```



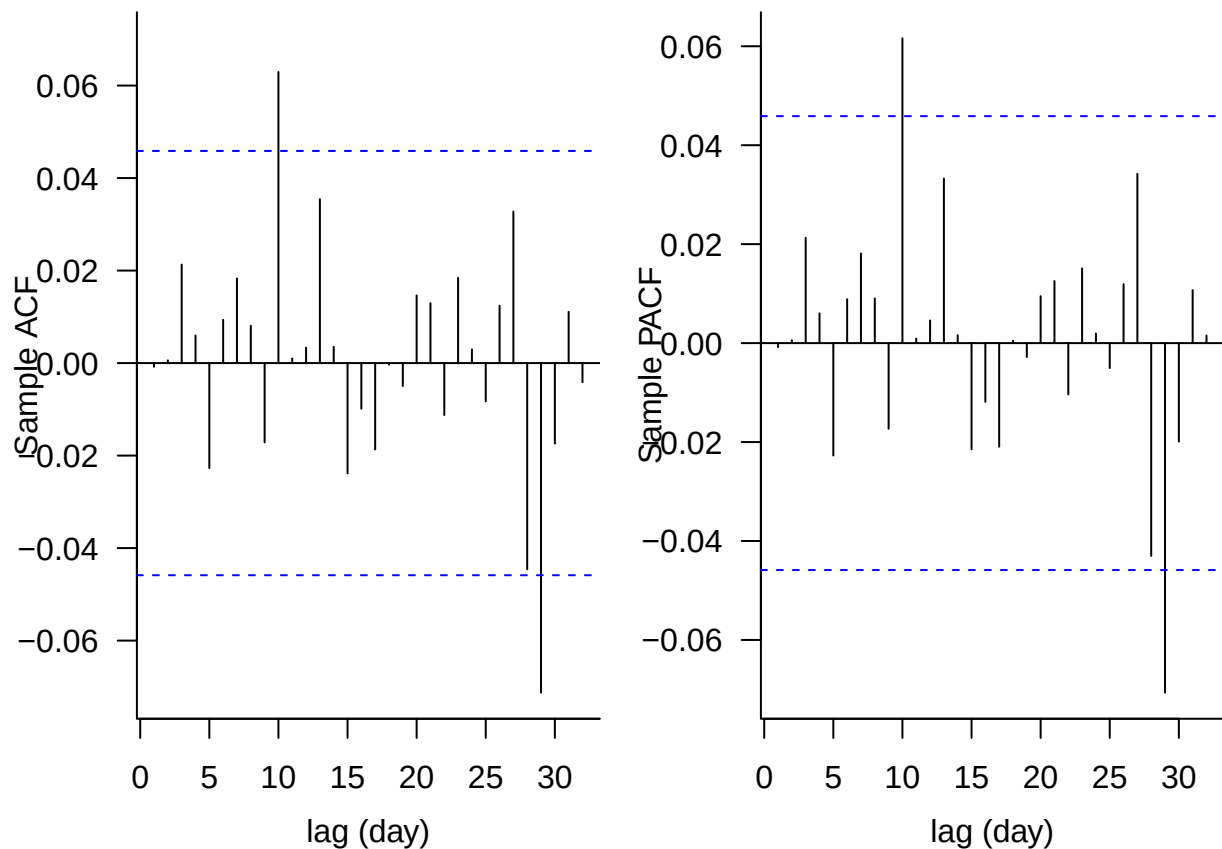
```

## extract the residuals
arma11.resids <- resid(arma11.model)
## time series plot of the residuals
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.2, 1, 0), mfrow = c(1, 2))
plot(year, arma11.resids, type = "l", xlab = "year",
     ylab = "ARMA(1,1) residuals", lwd = 0.6, col = "gray")
abline(h = 0, lty = 2)
## Normal Q-Q plot for the residuals
qqnorm(arma11.resids, main = "", cex = 0.75); qqline(arma11.resids)

```



```
## Sample ACF and PACF of the residuals
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.4, 1, 0),
    mfrow = c(1, 2))
Acf(arma11.resids, ylab = "Sample ACF", xlab = "lag (day)", main = "")
pacf(arma11.resids, ylab = "Sample PACF", xlab = "lag (day)")
```



```
## Carry out the Ljung-Box test
Box.test(arma11.resids, lag = 32, type = "Ljung-Box", fitdf = 2)
```

```
##
## Box-Ljung test
##
## data: arma11.resids
## X-squared = 32.757, df = 30, p-value = 0.3332
```

```
(arma21.model <- arima(sqrt.rosslare.ds, order = c(2, 0, 1)))
```

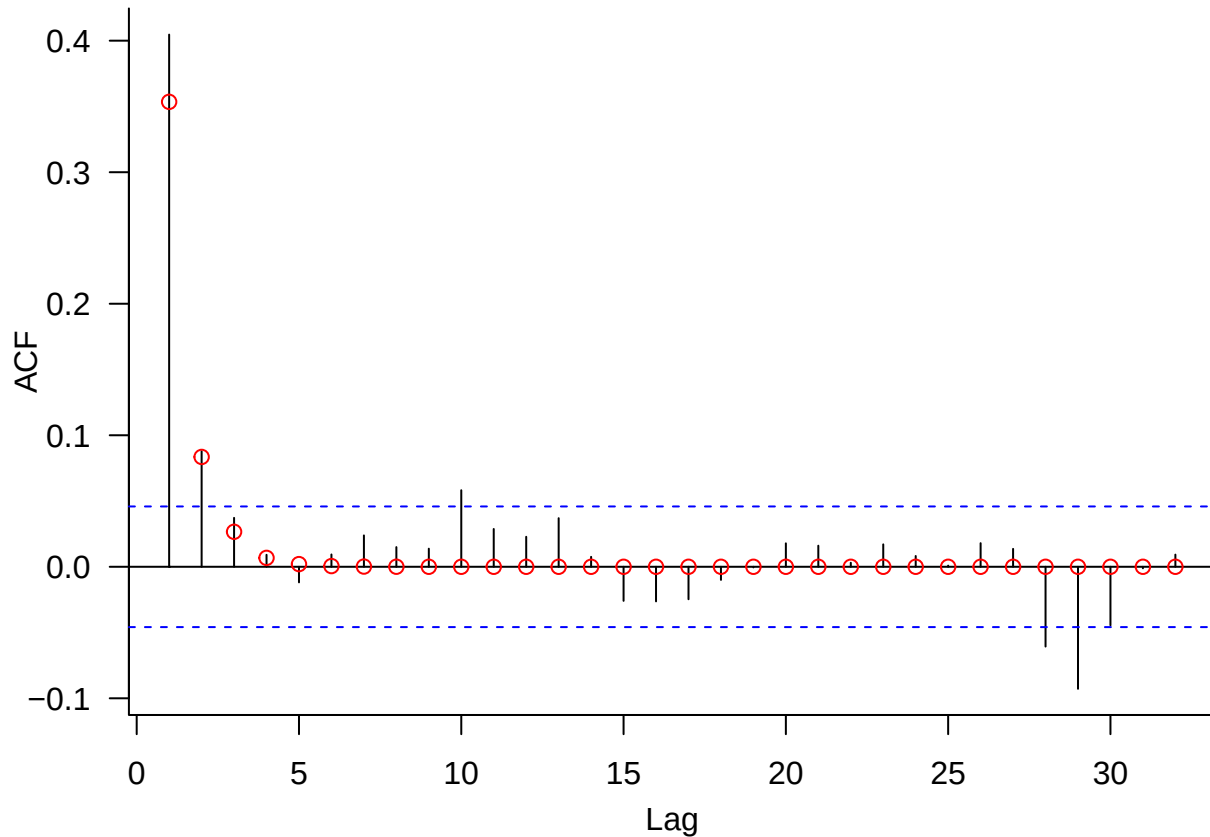
Fit an ARMA(2,1) model

```
##
## Call:
## arima(x = sqrt.rosslare.ds, order = c(2, 0, 1))
##
## Coefficients:
##          ar1          ar2          ma1  intercept
##         0.0703  0.0587  0.3768         3.3253
## s.e.  0.1691  0.0772  0.1663         0.0237
##
## sigma^2 estimated as 0.4107:  log likelihood = -1778.56,  aic = 3567.11
```

```

par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.2, 1, 0))
Acf(sqrt.rosslare.ds, main = "")
acf_true <- ARMAacf(ar = arma21.model$coef[1:2], ma = arma11.model$coef[3], lag.max = 32)[-1]
points(1:32, acf_true, col = "red")

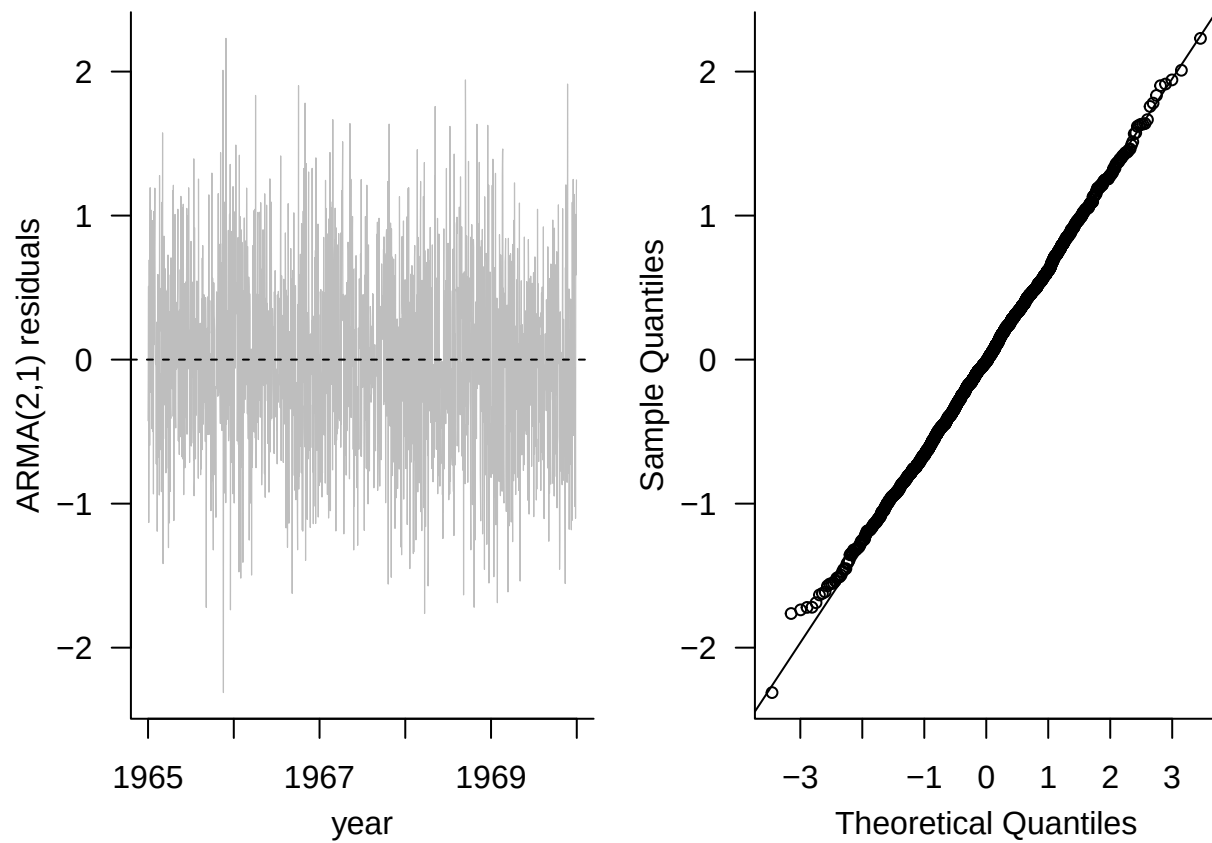
```



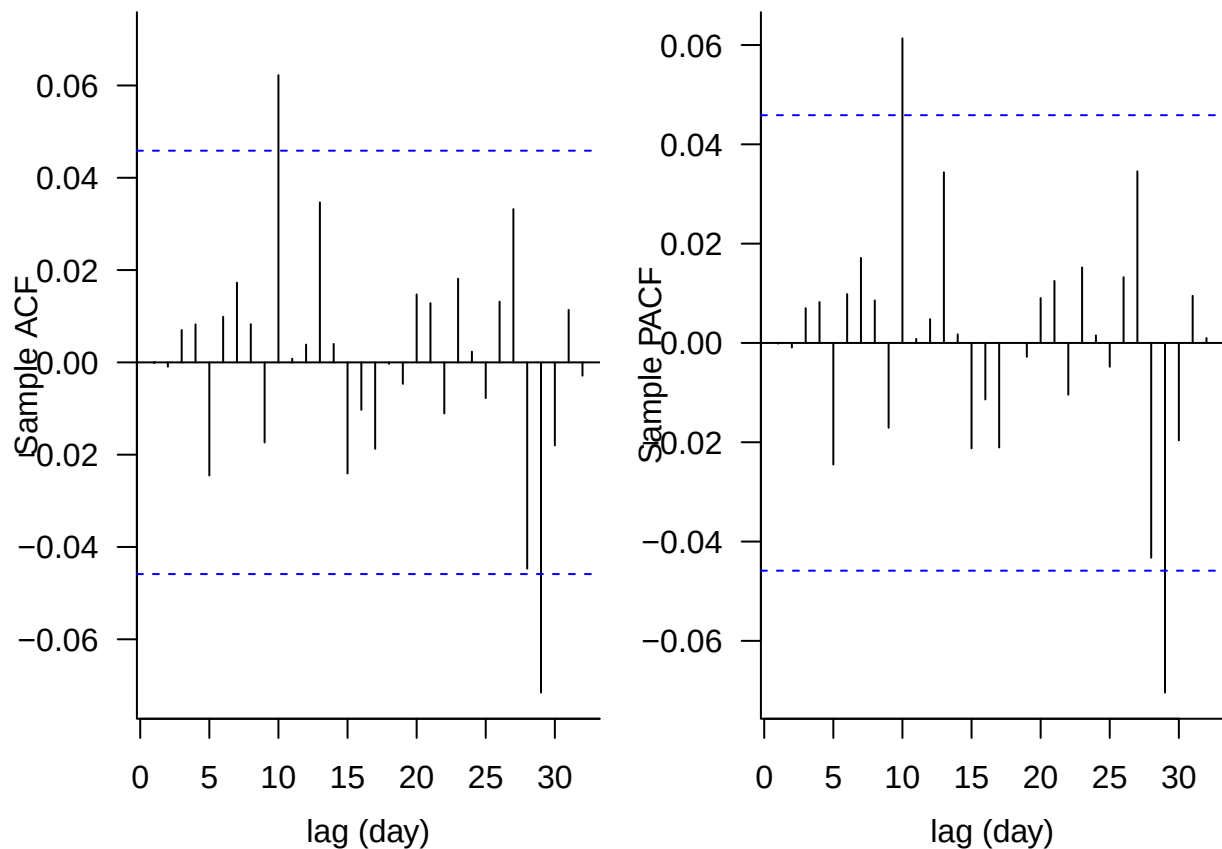
```

## extract the residuals
arma21.resids <- resid(arma21.model)
## time series plot of the residuals
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.2, 1, 0), mfrow = c(1, 2))
plot(year, arma21.resids, type = "l", xlab = "year",
     ylab = "ARMA(2,1) residuals", lwd = 0.6, col = "gray")
abline(h = 0, lty = 2)
## Normal Q-Q plot for the residuals
qqnorm(arma21.resids, main = "", cex = 0.75); qqline(arma21.resids)

```



```
## Sample ACF and PACF of the residuals
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.4, 1, 0), mfrow = c(1, 2))
Acf(arma21.resids, ylab = "Sample ACF", xlab = "lag (day)", main = "")
pacf(arma21.resids, ylab = "Sample PACF", xlab = "lag (day)")
```



```
## Carry out the Ljung-Box test
Box.test(arma21.resids, lag = 32, type = "Ljung-Box", fitdf = 3)
```

```
##
## Box-Ljung test
##
## data: arma21.resids
## X-squared = 32.171, df = 29, p-value = 0.3124
```

Use AIC to conduct model selection

```
AIC.to.AICC <- function(aic, n, npars) {
  aic - 2 * npars * (1 - n/(n-1-npars))
}
# calculate the length of the time series
n <- length(sqrt.rosslare.ds)

# Here are the AIC values
ar1.model$aic
```

```
## [1] 3581.432
```

```
ar2.model$aic
```

```
## [1] 3568.46
```

```
arma11.model$aic
```

```
## [1] 3565.642
```

```
arma21.model$aic
```

```
## [1] 3567.112
```

```
# convert the AIC values to AICC values.  
AIC.to.AICC(ar1.model$aic, n, 2)
```

```
## [1] 3581.438
```

```
AIC.to.AICC(ar2.model$aic, n, 3)
```

```
## [1] 3568.473
```

```
AIC.to.AICC(arma11.model$aic, n, 3)
```

```
## [1] 3565.655
```

```
AIC.to.AICC(arma21.model$aic, n, 4)
```

```
## [1] 3567.134
```

Based on the AIC (and AICC as well), we choose the ARMA(1,1) model.

Forecasting

```
## How many days will we predict into the future?  
h <- 10  
## Predict 'h' days into the future using the ARMA(1,1) model.  
sqrt.rosslare.forecast <- predict(arma11.model, h)  
sqrt.rosslare.forecast$pred; sqrt.rosslare.forecast$se
```

```
## Time Series:  
## Start = 1827  
## End = 1836  
## Frequency = 1  
## [1] 3.997161 3.458299 3.351724 3.330646 3.326477 3.325652 3.325489 3.325457  
## [9] 3.325451 3.325449
```

```
## Time Series:
## Start = 1827
## End = 1836
## Frequency = 1
## [1] 0.6409326 0.7022959 0.7045876 0.7046771 0.7046806 0.7046807 0.7046807
## [8] 0.7046807 0.7046807 0.7046807
```

```
## define the forecast variable
forecast <- sqrt.rosslare.forecast$pred
## The plus or minus value is the z critical value
## times the standard error for the forecast
me <- qnorm(0.975) * sqrt.rosslare.forecast$se
lower <- forecast - me
upper <- forecast + me
## Define the prediction time
fyear <- 1970 + (0:(h - 1)) / 365.25
```

Visualizing the Forecasts

```
# Display the three forecast stages in separate panels
par(bty = "l", mar = c(3.6, 3.6, 0.75, 0.6), las = 1, mgp = c(2.4, 1, 0),
    mfrow = c(3, 1), family = "serif")

# Plot the deseasonalized series from 1969 onward
plot(year[year > 1969], sqrt.rosslare.ds[year > 1969], type = "l",
     xlim = c(1969, max(fyear)), col = "grey", xlab = "year", ylab = "")

# Add forecasts and prediction intervals
lines(fyear, forecast, lwd = 2)
lines(fyear, lower, lty = 2, lwd = 2)
lines(fyear, upper, lty = 2, lwd = 2)

# Add the mean of the deseasonalized series
abline(h = mean(sqrt.rosslare.ds), lty = 3)
title("Forecasts for the deseasonalized square root wind speed")

# Add the estimated seasonal component back to the forecasts
adj.forecast <- fitted(harm.model)[1:h] + sqrt.rosslare.forecast$pred

# Adjust the prediction intervals
lower <- adj.forecast - me
upper <- adj.forecast + me

# Plot the square-root wind speed series from 1969 onward
plot(year[year > 1969], sqrt.rosslare[year > 1969], type = "l",
     xlim = c(1969, max(fyear)), col = "grey", xlab = "year", ylab = "")
title("Forecasts for the square root wind speed")

# Add forecasts and prediction intervals
lines(fyear, adj.forecast, lwd = 2)
lines(fyear, lower, lty = 2, lwd = 2)
```

```

lines(fyear, upper, lty = 2, lwd = 2)

# Back-transform forecasts and intervals to the original wind speed scale
plot(year[year > 1969], rosslare[year > 1969], type = "l",
      xlim = c(1969, max(fyear)), col = "grey", xlab = "year", ylab = "")
title("Forecasts for the wind speed")

# Add back-transformed forecasts and prediction intervals
lines(fyear, adj.forecast^2, lwd = 2)
lines(fyear, lower^2, lty = 2, lwd = 2)
lines(fyear, upper^2, lty = 2, lwd = 2)

```

